

Master Business Rules and Audit Specification

How money, stock and records work in NairaPlate

Version	1.4, 5 October 2026
Describes	The live system on 5 October 2026, including cash paid out of the drawer; and the database parts of offline Phase 0, ingredient and dish price history, paper-sale food cost at the sale time and paper entries (their screens are not yet released)
Readers	Parts 1 to 11: owners, accountants, managers. Appendices A to H: engineers and auditors
Database	52 tables, 72 access rules, 114 application functions, 56 triggers, read from the live database on 4 October 2026
Status	Reference manual. Sign-off is not yet possible: see Part 11

Contents

1. NairaPlate: Master Business Rules and Audit Specification
2. Part 1. The principles behind every money rule
3. Part 2. Roles: who can do what
4. Part 3. Area by area: the rules
5. Part 4. How the key numbers are worked out
6. Part 5. Audit: what is recorded
7. Part 6. Every kind of money event, accounted for
8. Part 7. The security model in plain words
9. Part 8. Evidence: what has actually been proved
10. Part 9. Known gaps and open items
11. Part 10. How the ledger programme got here
12. Part 11. Sign-off checklist for the security and safety sweep
13. Technical appendices
14. Appendix A. Tables and who can write to them
15. Appendix B. Functions callable from the app
16. Appendix C. Triggers
17. Appendix D. Ledger rules at a glance
18. Appendix E. Server routes
19. Appendix F. Audit events
20. Appendix G. SQL files for the programme
21. Appendix H. Glossary

NairaPlate: Master Business Rules and Audit Specification

Version 1.4, 5 October 2026

What is new in version 1.3.

1. **Cash paid out of the drawer (Step 8)** is live and has been used by a person on the real screens (3 October). A cashier can take cash out within an owner-set limit; above it the entry is a request the owner approves or declines; a shift cannot close while a request waits. See 3.7a.
2. **Catering deposits** now record cash or transfer (the database rule is live, Part B applied 3 October), and a cash deposit counts in expected cash. A confirmed order can be saved to any calendar. See 3.3.
3. **Supplier payments** now record how they were paid (four methods). The screen is live and has been used once; the older way of paying (no method) is not yet removed. See 3.4.
4. **Three database changes written by the assistant that builds the screens** were applied on 4 October 2026: save-once sales (20261030), dish price history (20261031) and paper (late) entries (20261101). Their screens exist in the source branch but **have not been released to the live site**. See 3.9, 3.10 and 3.11.
5. **The health check run on 4 October found a regression:** the late-entry script re-opened table privileges that the 3 October sweep had closed. A fix script is provided and not yet applied. See Part 7 and Part 9 (R1).
6. **Defects found by reading the unreleased paper-entry code against the live database** are listed in Part 9 (D1 to D6). Two of them stop a paper entry from being entered or approved. They should be fixed before the screens are released.

Version 1.2 (3 October 2026) corrected Part 4.5 (the sale-time cost snapshot already existed and was described wrongly in 1.1) and added the owner corrections for channel payouts, price decisions and batches (Step 7).

Version 1.1 recorded the sweep results: the batch fault (F0), the table locks A1 to A4 and the permission clean-up (B1 to B3). Version 1.0 (2 October 2026) described them as open.

This is the reference for how money, stock and records work in NairaPlate. It is written in two layers.

- **Parts 1 to 9** are for owners, accountants and managers. They are in plain language: what the rules are, who can do what, how a mistake is corrected, how the numbers are worked out, and what the system does and does not record.
- **Appendices A to H** are for engineers and auditors. They list the tables, access rules, functions, triggers, constraints, server routes and audit events, as read from the live database on 2, 3 and 4 October 2026.

How this document was produced, and how far to trust it

- The facts about the database (tables, access rules, functions, triggers) were read from the live database on 2, 3 and 4 October 2026, not written from memory.
- The formulas were read from the code (`src/lib`) and from the database functions.
- Every rule is marked with how it was proved, using these codes:
- **[C]** read from the live database or code.
- **[L]** tested on a local copy of the database during the build.
- **[R]** tested on the live database in a rehearsal (2 and 3 October 2026) that always undoes itself (Part 8).
- **[S]** the screen has not yet been used by a person in the Demo Kitchen test. A rule can be proved at the database level and still be unproven on the screen.
- **[P]** used by a person on the real screens in the Demo Kitchen (3 October 2026) and the result was checked in the database.

- Where something is not proved, or does not work, this document says so. Part 9 lists every known gap in one place.
- **Parts 3.9, 3.10 and 3.11 and the Phase 0 evidence were first drafted by the assistant that builds the app screens (4 October 2026).** I re-checked them against the SQL files, the code and the live database on 4 October and corrected what did not match. The corrections are listed in Part 8.6. Where this document says a script was applied, it was confirmed by reading the live database, not only by report.
- Money is stored in **kobo** (whole numbers). 100 kobo is ₦1. Times are Nigeria time (UTC+1).

Part 1. The principles behind every money rule

1. **Entries are added, never changed.** A payment, a write-off, a purchase, a refund or a count is a line in a record. Nobody edits or deletes that line afterwards. This applies to catering payments, customer credit, supplier entries, purchases, closed cash drawers and audit entries.
2. **A mistake is corrected by a new line beside the old one.** The correction is called a **reversal** (or an **adjustment** for a drawer count). The original stays on screen, struck through, with the reason for the correction next to it.
3. **Only owners reverse.** Reversals, write-offs, adjustments and closing someone else's shift need an owner (or a Supa Admin), and every one needs a **reason of at least 5 characters**.
4. **One home for every number.** Each figure that matters (expected cash, what a customer owes, what you owe a supplier, profit) is worked out in exactly one place. Every screen and report calls that place, so two screens cannot disagree.
5. **The database enforces the rules, not only the screens.** A screen hides a button. The database refuses the action. If the two ever disagree, the database wins.
6. **Money records are written only by approved functions.** The browser cannot insert or edit a sale, a payment, a purchase, a stock figure or a closed shift directly. It has to ask a database function that checks who is asking, which business they belong to, and whether the plan is active.
7. **Each business sees only its own records.** This is checked on every table and in every function, using the business in the signed-in person's login token.
8. **Nothing is guaranteed to be perfect.** These controls make fraud and mistakes hard and visible. They do not make them impossible. In particular, they cannot know whether a customer really paid, or whether cash was really counted. They can only make sure that what was recorded cannot be quietly changed.
9. **A sale is saved once.** Each sale attempt from the Till carries a code made on the device. If the same code arrives twice (a double tap, a lost reply), the database returns the first sale and makes no second one. (Applied on the database; not yet in the live Till. See 3.9.)
10. **A paper record is not a sale.** A sale written on paper during an outage becomes a sale only when an owner approves it, and it is then priced at the price in force when it really happened. (Applied on the database; not yet in the live site. See 3.11.)

Part 2. Roles: who can do what

NairaPlate has six roles. A person's role is set by the server when they sign in and is carried in their login token. It cannot be changed from the browser.

Role	In plain words
Owner	Runs the business. Can do everything for their own business, including corrections.
Supa Admin	A second manager. Treated the same as an owner for every action in this document. Only a Supa Admin can give or remove the Supa Admin role.
Cashier	Takes sales and payments at the till and runs the cash drawer. Cannot correct records.
Purchaser	Buys ingredients, records supplier payments, counts stock. Cannot correct records.
Cook	Logs batches and wastage, counts stock.
Platform Admin	NairaPlate staff. Approves businesses, manages plans and settings. Not part of any business.

2.1 Action matrix

"Yes" means the database allows it. A dash means it refuses. [C][R]

Action	Owner / Supa Admin	Cashier	Purchaser	Cook
Take a cash, split, transfer or credit sale	Yes	Yes	No	No
Void or refund a sale	Yes (any sale)	Yes, own sales only	No	No
Record a payment on a customer debt	Yes	Yes	No	No
Write off a debt, reverse a debt entry, add a debt by hand	Yes	No	No	No
Create a catering order, record a catering payment, change catering status	Yes	Yes	No	No
Reverse a catering payment	Yes	No	No	No
Log a purchase	Yes	No	Yes	No
Reverse a purchase	Yes	No	No	No
Change an ingredient's price	Yes	No	Yes	No
Record a supplier payment (a method must be chosen)	Yes	No	Yes	No
Reverse a supplier payment	Yes	No	No	No
Log wastage	Yes	No	Yes	Yes

Action	Owner / Supa Admin	Cashier	Purchaser	Cook
Log a batch	Yes	No	No	Yes
Submit a stock count	Yes (applies at once)	No	Yes (waits for an owner)	Yes (waits for an owner)
Approve or reject a stock count	Yes	No	No	No
Record the opening stock count	Yes	No	No	No
Open a cash drawer shift	Yes	Yes	No	No
Close their own shift	Yes	Yes	No	No
Close a shift someone else left open	Yes	No	No	No
Correct the count on a closed shift	Yes	No	No	No
Record a channel payout, decide a price	Yes	No	No	No
Take cash out of the open shift	Yes, any amount, any open shift	Yes, only on the shift they opened and within the limit; above it the entry is a request	No	No
Approve or decline a cash payout request, reverse a cash payout, set the cashier limit	Yes	No	No	No
Mark a purchase or a supplier payment "paid from the cash drawer"	Yes	No	Yes	No
Set or schedule a dish's selling price, cancel a scheduled price	Yes	No	No	No
Save a sale with a sale code, check whether a sale was saved	Yes	Yes	No	No
Submit a paper (late) entry	Yes	Yes	No	No
Approve, post or reject a paper (late) entry	Yes	No	No	No
Edit recipes	Yes	No	No	Yes
Manage staff and PINs	Yes (through the server)	No	No	No
Read the audit trail	Yes	No	No	No

Notes:

- Recipes can be edited directly by owners and cooks. Saving a new recipe version through the version function is owner only. [C]

- A cashier can void or refund only a sale they took themselves. A void is allowed only on the day of the sale. After that, use a refund. [C][R]
- A cook or a purchaser can submit a stock count, but it stays "pending" until an owner approves it. An owner's own count applies immediately. [C][R]
- Staff PINs are checked only on the server. The PIN hash and salt cannot be read or written from the browser by any role. [C][R]
- The save-once functions check the role only when they are about to make a new sale. A signed-in person of the same business who is not allowed to sell (a cook, a purchaser) and who already knows an existing sale code gets that sale's summary back. See Part 9 (S1). [C]

Part 3. Area by area: the rules

3.1 Sales at the till

What can be sold. Only dishes on the current menu, at the dish's current price. The price comes from the database, not from the screen. Since 4 October 2026 each dish's price is also kept as a history (see 3.10); the price a sale uses is the copy of today's price on the menu. [C][R]

Payment methods.

- **Cash.** The whole total is cash.
- **Transfer.** The whole total is a bank transfer. Automatic confirmation through Monnify exists, but is switched off by default for each shop and has **not** been proved end to end. [C][S]
- **Split.** Part cash and part transfer. The two parts must add up exactly to the total, and each must be more than zero.
- **Credit.** The customer owes the money. The sale creates a customer debt (see 3.2).

Saving a sale once and **paper entries** for outages are covered in 3.9 and 3.11. Both are applied on the database and are not yet in the live Till. [C]

Checks on every sale. The person must be a cashier, owner or Supa Admin. The plan must be active. There must be at least one item, each with a quantity above zero. The total must be above zero. Cash and transfer must add up exactly to the total. [C][R]

Stock. For dishes set to "made to order", each plate sold reduces the stock of its ingredients, using the recipe. Dishes set to "cooked in batches" reduce stock when a batch is logged instead. [C][R]

VOIDS and refunds.

Type	Rule
Void	Same day only. The whole sale is cancelled and its ingredients go back into stock. Not allowed on a part-refunded sale.
Part refund	More than ₦0 and less than what is left on the sale.
Full refund	Refunds what is left.

Every void or refund needs a reason of 5 or more characters, and is saved as its own record with who did it and why. The sale's status changes, but the record of the refund is never edited. [C][R]

3.2 Customer credit (money customers owe)

Each debt has an amount, a running total of what has been paid, and a running total of what has been written off. These totals are worked out from the entries. Nobody types them. [C][L]

Entry	Who	Rule
Payment	Cashier, owner	Cash or transfer. Part-payments are allowed. A payment can never be more than what is still owed.
Write-off	Owner only	Partial or the whole balance. Reason required. Shown as "Written off", never as "Paid".

Entry	Who	Rule
Reversal	Owner only	Undoes one payment or one write-off, once, for exactly the same amount. Reason required. A reversal cannot be reversed.
Debt by hand	Owner only	For money owed that did not come from a till sale. Cashiers create credit only by selling on credit at the till.

A debt is **settled** when nothing is left to pay (amount minus paid minus written off is zero). The size of a debt can never be changed after it is created. [C][L][R]

Debts that were marked settled before this system existed were given one "carried over" payment for their full amount. It is not known whether the money was actually received, and the history says so. [C]

3.3 Catering

A catering order has a contract total, a deposit, and later payments. Each deposit and each payment is its own entry. [C][L]

- **Reversal:** owner only, with a reason, for exactly the amount of the entry, once.
- **Payments and deposits** record whether they were **cash** or **transfer**. Since 3 October 2026 a deposit taken when the order is created must say how it was paid: an order with a deposit and no method cannot be saved. [C][R][P]
- A **cash deposit counts in the expected cash** of the shift that is open when it is recorded (see 4.1). The two deposits recorded before methods existed are marked "carried over", have no method and are not counted. [C]
- The database rule (a deposit needs a method unless it is carried over) applies to new entries only. Part B, which removed the old order function that took no method, was applied on 3 October 2026 and checked (0, 1, 1, 0). A catering order with a cash deposit was saved from the real screen the same day. [C][P]
- **Calendar entry (4 October 2026).** On a confirmed order the screen offers **Add to calendar:** a calendar file (.ics) that opens in Outlook, Google, Apple and others, a Google Calendar link, an Outlook link, and a WhatsApp message to the customer with the two links. It is built in the browser from the booking on screen. It has no server part and no database change. The entry holds the customer name, the items and the delivery address, runs at the event time (Nigeria time) for 2 hours (all day when no time is set), carries an alert one day before, and shows **no prices or balance**. [C][L] It has not yet been tried on a real phone or in a real Outlook account. [S]
- Orders carry a status (the statuses include enquiry, confirmed, delivered and cancelled). It is changed through a database function that cashiers, owners and Supa Admins can use. [C]

3.4 Supplier ledger (money you owe suppliers)

What you owe a supplier is worked out from entries only. [C][L]

Balance owed = credit purchases + payments that were reversed – payments – credit purchases that were reversed.

- A **negative** balance means the supplier owes you. This happens if you pay more than you owe (an advance). The screen asks for confirmation first: "This is more than you owe. The supplier will owe you the difference." [C][L][S]
- **Payment:** purchaser, owner. **Reversal of a payment:** owner only, with a reason.

- **Reversal of a credit purchase** (when a purchase is reversed, see 3.5) lowers the balance. If the supplier has already been paid part of it, the balance can go negative. The screen warns: "This reversal leaves the supplier owing you ₦X." [C][L][R]
- **How it was paid.** Since 3 October 2026 every new supplier payment records one method: **cash from the drawer**, **cash from somewhere else**, **bank transfer** or **other** ("other" needs a note of at least 5 characters). Only "cash from the drawer" creates a cash payout, and it needs an open shift (see 3.7a). The method cannot be edited afterwards. A fifth value, **legacy**, is written only by the older payment function for screens still open from before; it cannot be chosen on the new screen. Payments made before 3 October 2026 have no method and show "method not recorded". [C][R][P]
- **Not finished:** Part B (removing the older function that records no method) has not been applied, so a screen left open from before could still save a "legacy" payment. It is to be applied after the new screen has been used once more. [C]

3.5 Purchases, prices and stock

Logging a purchase. The purchaser or owner chooses the ingredient, quantity, unit, amount paid, how it was paid (cash, transfer or credit), a **grade** (A, B or C) and a **season** (plenty, normal or scarce). In one step the database: [C][R]

1. Converts the quantity into the ingredient's base unit (for example kg).
2. Works out the new price per base unit (amount paid divided by base quantity).
3. Updates the ingredient's current price, previous price, grade, season and stock.
4. Updates the price saved for that grade.
5. Saves the purchase, including a **snapshot of what the ingredient looked like just before** (price, grade, season, grade price) and how much stock this purchase added.
6. If it was bought on credit from a supplier, adds a supplier debt entry.
7. If the new price is more than 5% above the price for that grade, raises a price-spike alert for the owner.

Reversing a purchase (owner only, reason required). A purchase can be reversed only if **nobody has changed that ingredient's price since** (which also means it is the latest purchase of that ingredient). If a newer purchase or price change exists, the screen says: "Cannot be reversed because a newer price or purchase exists for this ingredient." [C][L][R]

A reversal:

- puts the ingredient back to the price, grade and season it had before, from the snapshot;
- puts the grade price back (or removes it if the purchase created it);
- takes the stock back out. **Stock is allowed to go below zero**, with a warning, because the kitchen may already have used it;
- lowers what you owe the supplier, if it was a credit purchase;
- adds a reversal line (negative quantity and amount) beside the original and an audit entry.

Purchases saved **before** 2 October 2026 have no snapshot and can never be reversed. [C][R]

An alert raised by a purchase stays after the purchase is reversed. It records what happened. [C]

Who can change stock and prices. Only the approved paths: purchases, reversals, sales, voids, wastage, batches, stock counts and the price-change function. **Nobody, including owners, can edit an ingredient's stock, cost, grade, season or price date directly.** An ingredient's unit cannot be changed once it has any purchase, stock history or stock-count line, and an ingredient with history cannot be deleted. [C][R]

Stock history. Every change to a stock figure writes a line to the stock trail with the reason. The reasons are: bought, purchase reversed, used in a sale, sale voided and put back, wastage, wastage entry removed, used in a batch, batch reversed and put back, stock-take correction, opening count, starting figure, and "changed by hand". "Changed by hand" should always be zero. As of 2 October 2026 it is zero. [C]

3.6 Wastage, batches and stock counts

- **Wastage:** cooks, purchasers and owners can log it. The database reduces stock. Owners can remove a wastage entry, which puts the stock back and is itself recorded in the stock trail. Wastage entries are a plain log, not a ledger, and an owner can edit or delete them. [C][R]
- **Batches:** cooks and owners log a batch of a dish that is set to "cooked in batches". The batch reduces stock for each ingredient. A dish set to "made to order" cannot have batches. Logging a batch was failing for everyone until 3 October 2026 because of a database fault (FO); the fix is applied and a batch of a normal dish was logged successfully in the rehearsal. A batch cannot be edited or deleted by anyone signed in. **A mistake is corrected by an owner reversing the batch** (reason of 5+ characters): the full ingredient quantities go back into stock, even if some has been used since, and the batch no longer counts in any total. A batch recorded before 3 October 2026 carries no stock trail and cannot be reversed. [C][R]
- **Stock counts:** counted amounts are compared with what the system expected. A difference needs a note. A cook's or purchaser's count waits for an owner to approve. Approval applies the counts as "stock-take correction" lines. [C][R]

3.7 Cash drawer (shifts)

A **shift** is one cashier's period at the till, with an opening float and a closing count.

- **One open shift per business at a time.** A second one cannot be opened until the first is closed. A handover is: close, then open. [C][L][R]
- **Opening** goes through a database function. **Closing** is done by the server, which works out the expected cash (see 4.1) and saves how it was worked out: cash sales, catering cash, debt cash and cash paid out. [C][L][R]
- If the count differs from the expected cash, an alert for the owner is raised. [C]
- **A closed shift can never be changed or deleted by anyone signed in.** [C][L][R]
- **Correcting a wrong count:** an owner adds an **adjustment** (reason required). The original count stays. The screen shows the original, each adjustment, and the adjusted result. The count can never be adjusted below ₦0. [C][L][R][S]
- **A shift cannot be closed while a cash payout request is waiting for the owner.** The database refuses every close, including an owner closing a shift someone left open. [C][R][P]
- **A shift left open:** an owner can close it with a reason. The expected cash is worked out up to that moment. The owner may leave the count empty, in which case the shift shows "not counted", raises no shortage alert, and cannot be adjusted. [C][L][S]

3.7a Cash paid out of the drawer (Step 8)

Cash that leaves the drawer for a market run, gas, transport or a supplier is recorded as an entry on the open shift, so an honest cashier is not shown as short. [C]

- **Who can take cash out.** The cashier who opened the shift (only on that shift), or an owner or Supa Admin on any open shift. A purchaser cannot take free-standing cash out, but can mark a purchase or a supplier payment "paid from the cash drawer". Cooks and visitors cannot. [C][R][P]

- **What is needed.** An open shift, an amount above zero, a category (market run, gas or fuel, transport, supplier settlement, other) and a note of at least 5 characters. Two more categories, "purchase" and "supplier payment", are set only by the system. [C][R]
- **The cashier limit.** The owner sets a limit for one shift (default **₦10,000.00**). A cashier's running total of their own direct payouts on the shift, plus the new amount, must stay within it. The total leaves out approved requests, owner entries, the two system categories and any payout already reversed. A payout that would pass the limit becomes a **request**. Owners have no limit. [C][R][P]
- **A request** does not count as cash out. It raises an alert for the owner and an audit line. The owner **approves** it (it then counts, as a payout linked to the request) or **declines** it with a reason of 5 or more characters (it never counts). A request can be decided once. [C][R][P]
- **Reversing a payout.** An owner can reverse a payout, with a reason of 5 or more characters, while its shift is open, once. The reversal is its own entry with a negative amount, so the cash counts as back in the drawer. After the shift has closed, the owner uses "Correct the count" instead. The database also allows reversing an approved payout; the screen shows the Reverse button only on payouts that were recorded directly, so an approved payout cannot be undone from the screen. [C][R][S]
- **Linked purchases and supplier payments.** A cash purchase or a supplier payment marked "from the cash drawer" is saved together with its payout, or not at all. A credit purchase cannot be paid from the drawer. Reversing the purchase or the payment reverses its payout in the same step while the shift is open; if the shift has already closed, the payout stays and an audit line says so ("cash payout left on closed shift"). [C][R][S]
- **Nobody can write the payout records directly.** Not owners, not the server key through the app: only the functions above. Entries cannot be edited or deleted. [C][R]
- **What a closed shift keeps.** The "paid out" figure is saved with the rest of the breakdown. [C][R]
- **Expected cash can go below zero.** In the Demo Kitchen test the shift opened with ₦2,000, took ₦39,000 of payouts and had no cash sales: expected cash was minus ₦37,000, the owner closed the shift with a counted ₦40,000, and the record shows an overage of ₦77,000. The system accepted it. [P]
- **Not yet used by a person:** reversing a payout, a purchase or supplier payment marked "from the drawer" (and reversing it), and closing a shift as balanced. [S]

3.8 Channel payouts and price decisions

- **Channel payouts** (money received from a delivery platform) are recorded by the owner. The database works out gross sales for the period and compares with what was received, and raises an alert for any difference. [C]
- **Price decisions** (publish, adjust portion, defer) are recorded by the owner. Publishing changes the dish's price, and since 4 October 2026 the new price is also added to the dish's price history (see 3.10). [C]
- Both tables are frozen: they can only be written by the app's own functions (`log_channel_payout` , `decide_price`) and cannot be edited or deleted by anyone signed in. [C][R]
- **Correcting a payout:** an owner reverses it (reason of 5+ characters). The reversal cancels the payout, which then shows as "Reversed" and no longer counts. The owner records the correct payout on the normal screen. A payout's mismatch alert is not linked to the payout, so the owner dismisses it by hand. [C][R][S]
- **Correcting a price decision:** an owner reverses it (reason of 5+ characters). For a "publish" decision the dish price goes back to what it was, but only if the dish still has the price that decision set and no later published decision is still standing. If the price has changed since, including by a hand edit to the dish, the reversal is refused. Adjust-portion and defer decisions changed no price, so only the record is reversed. [C][R][S]
- Each entry can be reversed once. A reversal is final and cannot itself be reversed. [C][R]

3.9 Saving a sale once, and safe degraded operation at the Till (Phase 0)

Phase 0 is **safe degraded operation, not offline selling**. No sale is saved, paid, taken from stock or treated as final until the server confirms it. The database part is applied (4 October 2026). The Till screens that use it are in the source branch and **are not on the live site yet**. [C]

The database rules (20261030_sale_once_a.sql). [C] - Each sale attempt carries a **sale code** (a random identifier made on the device). A business cannot have two orders with the same code; two businesses may reuse a code. - Three new functions, `create_cash_order_once`, `create_credit_order_once` and `create_transfer_order_once`, take the code. If the business already has an order with that code, they return **that** order and write nothing new. Otherwise they call the existing sale function unchanged and stamp the code on the new order. The existing three sale functions are not changed. - `find_sale_by_client_id` is read-only: it answers whether a sale with that code was saved. Only cashiers, owners and Supa Admins can use it, and only for their own business. - A sale with no code is refused. Two simultaneous sends of one code are made to wait for each other. - Before charging, the three functions bring any scheduled dish price that has started up to date (see 3.10). - **Known gap (S1)**: the three functions look up an existing sale before they check the caller's role, so a signed-in person of the same business who is not allowed to sell can read back a sale's summary if they already know its code. The code is a random identifier, so this is hard to exploit, but it is out of line with the rest of the design. See Part 9.

The device rules (Till screen). [C] - **Connection bar**. The Till shows when the connection is lost. It uses the browser's online and offline events and a small check of the server: every 25 seconds while online, then after a failure at 5, 10, 20 and then 30 seconds, each check giving up after 6 seconds. The check address (`/api/public/ping`) returns nothing and does no database work. When the Till is confirmed offline, final submission is disabled. - **Drafts**. A half-entered sale is kept on the device so a refresh or a dropped connection does not lose it. A sale whose save is uncertain stays locked until the server says whether it was saved ("Check again"). - **Privacy**. Customer name and phone are kept on the device only for a **credit-sale** draft. Cash, transfer and split drafts keep none. They are removed when the sale is saved, discarded or expires. - **Expired drafts**. A draft that was never sent and is more than 24 hours old is kept, shown as "Expired — not saved", and can never be charged. The cashier can print the paper form, ask the owner to review (this prints a review form; nothing is sent to the server) or discard it with a reason and a confirmation. If nobody acts, it is purged 7 days after expiry, leaving only a record of the sale code and the times, with no items or customer details. - **Paper form**. A printable form with the fields an owner needs to recover a sale later. It says it is not a saved NairaPlate sale. - **Not allowed during an outage**: refunds, voids, credit collection, supplier payments, purchases, stock counts, price changes, drawer adjustments, reversals and staff actions. - **Outage log**. The device keeps a short log of outage start and end times for 30 days.

Status. Database part applied and checked by the owner (1, 1, 6, false, false, 0). No sale has used a code yet (0 orders carry one). Not rehearsed on the live database. Module D of the Owner UAT script (UAT-OFF-01 to 15) has not been run by a person on a phone. [S]

3.10 Dish price history

A dish's selling price is kept as a history, so a later question ("what did this dish cost the customer last Tuesday?") has one answer. Applied on the database (20261031_dish_prices_a.sql , found applied on 4 October 2026). The screen (a price panel on the Recipes screen) is in the source branch and not on the live site. [C]

- **One row per price**. Each row has the price, the moment it starts and its source (starting price, owner, or recipe change). A price runs from its start until the next price starts, so periods cannot overlap or leave a gap. No two prices can start at the same moment. [C]
- **Add-only**. Started rows cannot be edited or deleted by anyone, including the server. A **scheduled** row can be cancelled, once, only before it starts. Nobody signed in can insert a row directly. [C]

- **Who.** Only an owner or Supa Admin can set a price (now, or from a future time; never in the past) or cancel a scheduled price, and the plan must be active. Each of these writes an audit line. [C]
- **History began on 4 October 2026** with one "starting price" row per current dish (8 rows live). Older sales keep the price on their own lines. Earlier prices are not reconstructed. [C]
- **Every other way a price changes is recorded automatically.** A rule on the recipes table adds a row whenever a recipe's price changes by any route: saving a recipe, a published price decision, the reversal of a decision (Step 7), or an owner's direct edit. [C]
- **The menu price is a copy of today's price.** `recipes.selling_price_kobo` is kept in step with the history. A scheduled price reaches that copy **when a sale or a screen next touches the business**, not on a timer; the three save-once sale functions do this before charging. [C]
- **Each new order line records which price row it used** (`order_items.dish_price_id`). Sales before the change have no link. [C]
- **The lookup** `dish_price_at(dish, time)` is the only way to ask what price applied at a moment. It works on the dish identifier, which all saved versions of one dish share. It does **not** work on a single recipe version's own identifier (see Part 9, D1). [C]
- **Limits.** Ingredient prices still have no history. A sale's plate cost is still the cost frozen at the moment it was saved (4.5). [C]
- **Status.** Table, functions and rules are on the live database; 0 order lines yet carry a price link; not rehearsed on the live database; the screen has not been used by a person. [S]

3.11 Paper (late) entries

A **paper (late) entry** recovers a sale that was written on paper during an outage. Applied on the database (`20261101_late_entries_a.sql` , applied 4 October 2026 and confirmed by reading the live database; 0 entries so far). The screens (`/late-entries` , a link on the home screen, a label on the Orders screen) are in the source branch and **are not on the live site**. The rules below are read from the SQL. They have **not** been run. Part 9 lists six defects found in them (D1 to D6), two of which stop an entry from being entered or approved. [C]

- **A paper record is not a sale.** Submitting one changes no sales, cash, stock or report. Only an owner's approval posts a sale. [C]
- **Submitting** (cashier, owner, Supa Admin; plan active). Needs: a paper reference of at least 2 characters; an outage reason of at least 3; the real time of the sale, not in the future (5 minutes of grace) and **not more than 72 hours ago**; payment cash, transfer or split; at least one item; cash plus transfer equal to the total; a split with both parts above zero. Each item is priced from the dish's price history **at the real sale time**, and the price is frozen on the entry. Resubmitting the same sale code returns the first entry. [C]
- **Status on entry.** "Submitted", or "needs shift review" when no shift covered the sale time or that shift is closed. [C]
- **Rejecting** (owner, Supa Admin): a reason of 5 or more characters, once, only while submitted or awaiting shift review. [C]
- **Approving and posting** (owner, Supa Admin; plan active). If the entry needs shift review, or no shift is open now, the owner must choose how the cash is treated: **"closed shift, already included"** (the cash was in that shift's count) or **"closed shift, late cash"** (the cash was not in the count; the database adds a count adjustment for the cash amount to that shift). Otherwise the entry is posted into the open shift. Posting creates a normal order marked as a late entry, carrying the paper reference, the real sale time, the delay, who entered it and who approved it, with its lines at the frozen price; stock is reduced at that moment for made-to-order dishes; an audit line is written. [C]
- **Frozen.** Entries and their lines cannot be edited or deleted by anyone signed in; only the three functions change them. [C]

- **Not allowed through this route:** correcting or cancelling an earlier sale, voids and refunds. Those stay online-only. [C]
- **Roles.** Cashiers, owners and Supa Admins can read entries of their own business. [C]
- **Status.** Applied; no entry exists; not rehearsed; not used by a person. **Do not release the screens until D1 to D4 are fixed.** [S]

3.12 Ingredient price history and paper-sale food cost

Added in version 1.4. Applied on the database (`20261103_ingredient_prices_a.sql` and `20261105_late_entry_cost_at.sql` , both read live on 5 October 2026). The screens are in the source branch and are **not on the live site**. Not rehearsed, not used by a person. [C]

- **What is kept.** Every ingredient price is added to an add-only list with the time it started, who or what set it (a purchase, a purchase reversal, a manual price change, or a labelled backfill) and the grade track it belongs to (current, A, B or C). Rows cannot be edited or deleted. No signed-in person can read the table directly; they read it through two checked functions open to owners, Supa Admins and purchasers of the same business. [C]
- **Writers.** A rule on purchases and the re-created `set_ingredient_price` . A rule refuses, at the end of the save, any price change from the app that left no history row. [C]
- **Price at time T.** The latest row on or before T, on the exact track. A price of zero means "no price then". If there is no row yet the answer is "unavailable before history". No other grade's price and no later price is ever used. [C]
- **Food cost of a paper sale.** The database works out each plate's cost from the ingredient prices in force at the real sale time. A cost sent by an app is never trusted. If any one ingredient has no price at that time, the whole plate is "unknown": the owner holds the entry, or approves it at today's cost with a reason of at least 5 letters, and that line is stored labelled "estimated (today's prices)" with who decided, when, why and which ingredients were missing. Till sales are unchanged. [C]
- **What the live database showed on 5 October (read only).** Both check scripts returned exactly their expected values. The history table holds 11 rows, all labelled backfill (one per priced ingredient; 8 are "known from the migration only"; 3 are dated from the ingredient's last price change; the earliest is 25 September 2026). 11 older purchases were skipped because they had no base quantity. For all 8 current dishes the cost at "now" equals today's cost, and for 1 September 2026 every dish is "unknown", never guessed. [C]
- **Limits.** Sales before 25 September 2026 cannot be costed at their sale time and will need the owner's estimate. No grade prices exist yet (0 rows). Batch and wastage costs are still worked out in the browser and sent to the database (read in `src/routes/batches.tsx`) (see Part 9, S4). [C]

Part 4. How the key numbers are worked out

4.1 Expected cash in a drawer

Expected cash = opening float + cash part of every paid or part-refunded cash or split sale in the shift – part refunds on those sales (never more than the cash taken on that sale) + cash catering payments and cash catering deposits in the shift + cash debt payments in the shift – cash paid out of the drawer in the shift (payouts minus reversals)

- Voided and fully refunded sales add nothing. [C]
- A catering or debt payment that is reversed has a matching minus entry with the same method, so a payment and its reversal inside the same shift net to nothing. A reversal made in a later shift counts in that later shift. [C][L]
- **Cash paid out.** A payout, or a purchase or supplier payment marked "paid from the cash drawer", reduces expected cash. A **waiting request** and a **declined request** do not. A payout that was reversed nets to nothing. [C][R][P]
- The shift window runs from the moment the shift opened to the moment it closed. Orders and payments belong to the shift by the moment they were **recorded**. [C]
- **Not counted:** the transfer part of any sale; catering deposits recorded before methods existed (the two carried-over ones); and any cash purchase or supplier payment that was **not** marked "paid from the cash drawer" (it was paid from somewhere else). [C]
- **A paper entry posted later** is recorded when the owner approves it, so its cash falls into whichever shift is open at that moment, even if the sale really belonged to an earlier shift. See Part 9, D3. [C]
- Expected cash can be negative (3.7a). [P]
- The count the cashier types is compared with this number. **Difference = counted – expected.** After an owner adjustment, the adjusted difference = counted + adjustments – expected. [C][L]
- The calculation lives in one place (`src/lib/cash-drawer.ts`) and is used by closing a shift, by the owner's cash forecast, and by closing an abandoned shift. [C]

4.2 What a customer owes

Owed = debt amount – payments – write-offs (payments and write-offs both after reversals). Never below zero.

4.3 What you owe a supplier

See 3.4. A negative result means the supplier owes you.

4.4 Ingredient price and plate cost

- **Price per base unit** from a purchase = amount paid ÷ quantity in the base unit. [C]
- **Plate cost** of a recipe = for each ingredient, its recipe quantity (converted to the base unit) × its price, added up, divided by the number of portions the recipe makes. If the recipe is set to a cost grade, the price for that grade is used; otherwise the ingredient's current price. If a quantity cannot be converted to the ingredient's unit, no cost is given. [C]
- Each sold line keeps the plate cost at the time of sale. [C]

4.5 Profit and loss

- **Gross sales** (paid orders), **cost of goods** (recipe cost of plates sold, plus wastage), **gross margin** and **food cost percentage** come from one calculation (`src/lib/pnl.ts`). [C]
- **Cost frozen at the moment of sale.** Since 1 October 2026 every sold line stores what one plate cost at that moment (`order_items.cost_per_plate_kobo` , set by the database from the recipe version and its cost grade). A signed-in person cannot change it afterwards ("The cost of a sold item cannot be changed"). The profit and loss uses that stored figure as it is, so later price rises do not rewrite earlier sales. [C]
- **Known limitation:** sold lines from before 1 October 2026 carry no stored cost. They are costed with each ingredient's **current** price, and the result counts them as "estimated". In the live data on 3 October 2026, 107 of 110 sold lines are in this group, because almost all are older test sales; all 3 lines since the change carry a frozen cost. Ingredient prices themselves are still not versioned, so a restated cost for old sales is not possible. Wastage entries store their own cost when logged. [C]

4.6 A dish's selling price at a moment in time

Price at time T = the price from the dish's price history whose start is the latest one on or before T and that was not cancelled.

This is the only definition (`dish_price_at`). It is used to price paper (late) entries at the real sale time. From 4 October 2026 each new order line also records which price row it used. Sales before then have no link and no history before 4 October exists. See 3.10. [C]

Part 5. Audit: what is recorded

5.1 The audit trail

Every important action writes a line to the audit trail with: **who** (the person and their role), **what** (the action), **which record**, a short description, and **when**. The audit trail can be read only by owners and Supa Admins of that business. [C]

Audit lines are written by database functions, by database triggers, or by the server. [C]

Events the system writes (Appendix F lists them all):

Area	Events
Sales	order adjusted (void or refund)
Customer credit	credit created by hand, payment recorded, written off, entry reversed
Catering	order created (says the deposit method), payment recorded, payment reversed
Supplier	payment recorded (says how it was paid), payment reversed
Purchases and stock	cost changed, purchase reversed, stock count submitted, approved, rejected
Cash drawer	drawer opened, shortage or overage at close, count adjusted, shift closed by owner; cash payout recorded, requested, approved, declined, reversed or left on a closed shift; cashier limit changed
Paper entries	late entry submitted, rejected, posted
Prices	price published; dish price started, scheduled or cancelled
Business and staff	business created, approved, rejected, suspended, reactivated; staff created, role changed, deactivated, PIN reset; sign-in success, failure, account locked
Platform	settings changed, subscription payment recorded, payment connection, payment mode changed

On 3 October 2026 (end of day) the live audit trail holds 153 lines across 29 event types, up from 118 lines and 16 types on 2 October. The cash payout events, the cashier limit change and the forced close all appear in it from the real screens. [C][P]

5.2 What is not audited

Being honest about gaps matters more than a long list:

- **A purchase being logged** has no audit line of its own. The purchase record is the evidence, and a price change writes a "cost changed" line.
- **Wastage and batch entries** have no audit line.
- **A shift closing with no difference** has no audit line. The shift record itself holds the figures. Opening a shift does.
- **Sales** are recorded in the orders themselves, not in the audit trail. A sale saved through a sale code leaves no extra line.

-
- **A dish price copied onto the menu** by a recipe save, a price decision or an owner's direct edit is recorded as a price history row, not as an audit line. An owner setting, scheduling or cancelling a price does write an audit line.

5.3 Protection of the audit trail itself

- Nobody can edit or delete an audit line. [C][R]
- Nobody signed in can *add* an audit line either. Lines are written only by database functions and server routes, so a cook can no longer pad or forge the trail (this was gap A2, closed on 3 October 2026). [C][R]

Part 6. Every kind of money event, accounted for

For each kind of transaction: where it is recorded, what writes it, who may do it, how it is undone, what audit line it leaves, and whether it counts in the cash drawer's expected cash.

#	Event	Recorded in	Written by	Who	How it is undone	Audit event	In expected cash?
1	Cash sale	orders, order lines, stock trail	<code>create_cash_order</code>	Cashier, owner	Void (same day) or refund	none for the sale; "order adjusted" when voided	Yes
2	Split sale	same	<code>create_cash_order</code>	Cashier, owner	same	same	Cash part only
3	Transfer sale	orders, payment requests	<code>create_transfer_order</code> ; provider webhook marks it paid	Cashier, owner; server	Void or refund	same	No
4	Credit sale	orders + customer debt	<code>create_credit_order</code>	Cashier, owner	Debt write-off or reversal	credit events	No
5	Void	order adjustments, order status, stock put back	<code>adjust_order</code>	Cashier (own), owner	Not reversible: it is final	order adjusted	Removes the sale
6	Part or full refund	order adjustments, order status	<code>adjust_order</code>	Cashier (own), owner	Final	order adjusted	Reduces cash by the refund, up to the cash taken
7	Catering order and deposit	catering order, catering payment (deposit, with its method)	<code>create_catering_order</code> (12-argument version)	Cashier, owner	Reverse the deposit entry	catering order created	Yes, if the deposit is cash (since 3 October 2026)
8	Catering payment	catering payments	<code>record_catering_payment_v2</code>	Cashier, owner	Owner reversal	catering payment recorded or reversed	Yes, if cash
9	Customer debt payment	credit payments	<code>record_credit_payment</code>	Cashier, owner	Owner reversal	credit payment recorded	Yes, if cash
10	Debt write-off	credit payments	<code>write_off_credit</code>	Owner	Owner reversal	credit written off	No
11	Debt by hand	customer credits	<code>create_manual_credit</code>	Owner	Write-off	credit created by hand	No
12	Supplier credit purchase	supplier entries, purchase	<code>log_purchase</code>	Purchaser, owner	Purchase reversal	none (price change line)	No

#	Event	Recorded in	Written by	Who	How it is undone	Audit event	In expected cash?
13	Supplier payment	supplier entries (with its method)	record_supplier_payment_v2 (the older record_supplier_payment until part B)	Purchaser, owner	Owner reversal	supplier payment recorded or reversed	Only if paid from the drawer (then it is also event 31)
14	Cash or transfer purchase	purchases, stock, price	log_purchase , or log_purchase_from_drawer	Purchaser, owner	Purchase reversal	cost changed	Only if paid from the drawer (then it is also event 31)
15	Purchase reversal	purchases, stock, price, supplier entry	reverse_purchase	Owner	Final	purchase reversed	No
16	Price change	ingredient price, grade price	set_ingredient_price	Purchaser, owner	Another price change	cost changed	No
17	Wastage	wastage log, stock trail	direct entry; database adjusts stock	Cook, purchaser, owner	Owner removes the entry	none	No
18	Batch	batches, stock trail	log_batch	Cook, owner	Owner reversal (event 29)	none	No
19	Stock count	stock counts, lines, stock trail	submit_stock_count , decide_stock_count	Cook, purchaser, owner; owner approves	A new count	stock count submitted, approved, rejected	No
20	Shift open	cash drawers	open_cash_drawer	Cashier, owner	n/a	drawer opened	n/a
21	Shift close	cash drawers	server route	Cashier (own), owner	Count adjustment	shortage or overage	n/a
22	Shift closed by owner	cash drawers	server route	Owner	Count adjustment (if counted)	shift closed by owner	n/a
23	Count adjustment	cash drawer adjustments	adjust_closed_drawer	Owner	Another adjustment	count adjusted	n/a
24	Channel payout	channel payouts, alert	log_channel_payout	Owner	Owner reversal (event 27)	none	No
25	Price decision	price decisions	decide_price	Owner	Owner reversal (event 28)	price published	No

#	Event	Recorded in	Written by	Who	How it is undone	Audit event	In expected cash?
26	Subscription payment	subscription payments	server only	Platform	n/a	subscription payment recorded	n/a
27	Payout reversal	channel payouts (negated row)	reverse_payout	Owner	Final	payout reversed	No
28	Price decision reversal	price decisions; dish price	reverse_price_decision	Owner	Final	price decision reversed	No
29	Batch reversal	batches (negated row), stock trail	reverse_batch	Owner	Final	batch reversed	No
30	Cash payout (direct)	cash drawer payouts	record_cash_payout	Cashier (own shift, within the limit), owner	Owner reversal while the shift is open	cash payout recorded	Yes, reduces expected cash
31	Cash payout from a purchase or a supplier payment	cash drawer payouts, linked to the purchase or payment	log_purchase_from_drawer , record_supplier_payment_v2 (method cash from the drawer)	Purchaser, owner	Reversing the purchase or payment reverses it (shift open)	cash payout recorded; cash payout reversed or left on closed shift	Yes
32	Cash payout request	cash drawer payouts (request)	record_cash_payout	Cashier over the limit	Owner approves (33) or declines (34)	cash payout requested	No until approved
33	Request approved	cash drawer payouts (payout linked to the request)	approve_cash_payout	Owner	Reversal (the database allows it; the screen does not show it)	cash payout approved	Yes
34	Request declined	cash drawer payouts (decline)	decline_cash_payout	Owner	Final	cash payout declined	No
35	Cash payout reversal	cash drawer payouts (negated row)	reverse_cash_payout , or automatically with a purchase or supplier reversal	Owner	Final	cash payout reversed	Puts the cash back
36	Cashier limit change	drawer settings	set_drawer_payout_limit	Owner	Another change	drawer limit changed	Changes what needs approval
37	Dish price change	dish price history (and the menu copy)	set_dish_price , cancel_dish_price , and a rule on recipes	Owner	Cancel before it starts, or set another price	dish price started, scheduled or cancelled	No

#	Event	Recorded in	Written by	Who	How it is undone	Audit event	In expected cash?
38	Paper entry submitted	paper entries and their lines	submit_late_entry	Cashier, owner	Owner rejects	late entry submitted	No
39	Paper entry rejected	paper entries	reject_late_entry	Owner	Final	late entry rejected	No
40	Paper entry posted as a sale	orders (marked late), order lines, stock; possibly a count adjustment	approve_and_post_late_entry	Owner	Void or refund like any sale	late entry posted	Cash part: yes, in the shift open at approval (see D3)
41	Sale saved with a sale code	orders (as events 1 to 4)	create_cash_order_once , create_credit_order_once , create_transfer_order_once	Cashier, owner	As the sale	as the sale	As the sale

Events 40 and 41 exist on the database and are not yet used by the live site. Event 7 now counts a cash deposit; event 13 and 14 count only when paid from the drawer.

Events that do not fit the table: staff changes and PIN resets (server routes, audited); business sign-up and approval (audited); daily summary emails and reminders (system, logged separately). [C]

Part 7. The security model in plain words

Two ways in. The app talks to the database in two ways. [C]

1. **From the browser**, using the person's login. The database checks the login on every request using row security: a person sees and changes only their own business's rows, and only what their role allows.
2. **Through server routes**, which check the login token or a secret first and then use a stronger key. These are used where the browser must not be trusted: signing in with a PIN, closing a shift, managing staff, connecting a payment provider, the scheduled emails and the payment webhook. Appendix E lists all 17.

The locks, in layers.

- **Row security** on all 52 tables (50 app tables and 2 private backup tables). No rule lets a signed-out visitor read or write anything. [C][R]
- **Write functions.** Money tables, refund records, the audit trail, payouts, price decisions and batches have no direct write access for signed-in people. Only database functions, which check the role, the business and the plan, can write. [C][R]
- **Guard triggers.** Even where a write path exists, a trigger on the table refuses edits and deletes of finished records, and refuses direct writes to protected columns. [C][R]
- **Fixed search path.** All 91 database functions that run with extra privilege have their search path fixed, so they cannot be tricked into using a different table. [C]
- **Secrets** (payment provider keys, the scheduler secret) are stored in the database vault and never returned to the browser. [C]

A gap in this layer since 4 October 2026. The health check (a read-only query) was run on the live database on 4 October after the paper-entry script was applied. Three of its first eight values were wrong: visitors hold 21 table privileges (expected 0), signed-in people hold 9 risky privileges (TRUNCATE, TRIGGER, REFERENCES; expected 0), and visitors can run 2 trigger functions (expected 0). All of it comes from the two new tables `late_entries` and `late_entry_items`, a view `dish_price_periods`, and the guard function `late_entries_protect`, which were created with the database's default grants. Row security is on for both tables with only a read rule, so a visitor or a signed-in person still cannot write through the app, which is why nothing was exposed. It is nonetheless the same kind of gap that B1 to B3 closed on 3 October and it breaks the sweep rule. A fix script (`20261102_hygiene_after_late_entries.sql`) is provided, tested on a local copy, and **not yet applied**. See Part 9 (R1). [C]

Staff sign-in. PINs are hashed. The server is the only place a PIN is checked. A signed-in person's role and business come from the server and cannot be set from the browser. Businesses must be approved. A business that is suspended or whose plan has ended is locked out. Owners can still sign in to see the locked screen. [C]

What the controls do not cover is listed in Part 9.

Part 8. Evidence: what has actually been proved

8.1 Automated tests

- **290 automated checks across 28 test files** pass on the released code (`main` , 4 October 2026). **302 checks across 30 test files** pass on the source branch, which also holds the offline, price history and paper-entry code. They cover the shared calculations (balances, reversals, expected cash, adjustments, calendar entries, payment methods, labels and rules shown on screens). [L]
- For every step of the ledger programme, the database changes were tested on a local copy of the database before release: the loophole shown first, then every allowed action, every refusal, other businesses, every role, repeated runs, and the rollback. [L]

8.2 The live rehearsal, 2 and 3 October 2026

A script acted as each role in the Demo Kitchen business, using the real database functions and rules, and then always ended with a deliberate error so that nothing was saved. [R]

- **First run, 2 October (127 steps):** 107 passed, 9 were flagged, 11 were information only. The flags were six genuine gaps (A1, A2, A3, A4), two steps caused by the batch fault (FO), and one false alarm (the test set a business status to the value it already had).
- **Run after FO, B1 to B3, A1 and A2 were applied:** 114 passed, 6 flagged, 0 test errors. The 6 were exactly A3 and A4.
- **Final run, 3 October, after A3 and A4 were applied:** 119 passed, 1 flagged, 0 test errors, plus information rows. The one flag was again the false alarm of setting a status to its current value. The test was corrected to use a different value, and that step was re-run: an owner trying to change their own business status is refused ("Only the platform admin can change a business's status").
- **Passed (examples):** a cashier took a cash sale and voided it; part and full refunds worked and over-refunds were refused; cooks, purchasers and visitors were refused where they should be; a purchaser logged a credit purchase and an owner reversed it, restoring the price and the supplier balance; supplier, customer-credit and catering payments and reversals followed the role rules; stock counts needed owner approval; shifts opened once and could not be opened twice; closed shifts could not be rewritten; an owner could add a count adjustment; another business could see none of Demo Kitchen's data and could not act on it; visitors could read and write nothing; no owner could write, edit or delete refund records, audit lines, payouts, price decisions or batches directly; the real functions for a price decision and for a batch of a normal dish still worked.
- **By design, still refused:** a batch of a "made to order" dish; automatic transfer sales in a business that has not switched them on.
- **Not saved:** the rehearsal changed nothing.
- What the rehearsal cannot show: it tests the database as each role, not a person using the screens.

8.2b The correction functions, 3 October 2026

- **Local database copy:** refusals for a cashier, cook, visitor, another business's owner and an expired plan; short reasons; double reversals; forged reversal rows; the price chain (reversing an older decision is refused while a newer one stands, and works after the newer one is reversed); a price edited by hand blocks reversal; stock put back in full after later use; a batch whose dish was later made "made to order" can still be reversed; rollback and re-apply. [L]

- **Live rehearsal of the correction functions:** 37 passed, 0 findings, 0 test errors. Stock went 5.20 to 5.19 on logging a batch and back to 5.20 on reversal. The data was checked afterwards and nothing was left behind. [R]
- **Full rehearsal with the correction steps included:** 143 passed, 0 findings, 0 test errors. [R]
- 246 automated checks pass (10 are new for the corrections). [L]

8.2c Cash out and supplier method rehearsals, 3 October 2026

- **After Step 8 part A was applied** and before its screens were released: a live rehearsal of **185 steps passed, 0 findings, 0 test errors**, including 42 new cash-out steps (roles, the running-total limit, requests, approval, decline, the close guard, direct writes refused, linked purchase and supplier payments in both directions, the catering method). Nothing was left behind. [R]
- **After the supplier payment method was applied:** a live rehearsal of **200 steps passed, 0 findings, 0 test errors**, including 15 supplier-method steps (the four methods, a missing or made-up method, "other" without a note, the older "legacy" value refused on the new function, the internal writer closed to signed-in people, the method not editable, a drawer payment and its payout reversed together). Afterwards the data was checked: no payments carried a method, no shift was open and the payout count was unchanged. [R]
- **Local database copy:** the same cases, plus the older functions still working for open screens, part B and its rollback, and the rollback refusing once payments carry a method. One real defect was found and fixed before release: a payment with no method at all slipped past the method rule, because an empty value passes a check. [L]
- **Part B for catering** (the old order function removed, the deposit rule added) was checked live: old function 0, new function 1, rule 1, deposits without a method 0. [C]

8.2d What people did on the real screens, 3 October 2026

A person used the Demo Kitchen on a phone and the result was checked in the database. [P]

Screen	What was done	What the database shows
Batches	Logged a batch of a dish, reversed it with a reason	Batch marked reversed; the ingredient stock trail shows "used in a batch" then "batch reversed, put back"; stock back to 5.2 kg
Channel payouts	Saved a payout, reversed it	Reversed, struck through, no Reverse button left; one "payout reversed" audit line
Price decision history	Reversed a published decision	A reversal row was added; the price on that recipe version went back from ₦676.93 to ₦646.16; the live ₦1,500 version was untouched
Cash drawer	Cashier took ₦5,000 twice, was told the limit was used up, asked for ₦4,000	The ₦4,000 became a request; the Close shift button was greyed out
Cash drawer	Owner declined one request, approved another, took ₦20,000 directly, changed the limit to ₦12,000 and back	Approved, declined, direct payout and two limit-change audit lines all present
Cash drawer	Owner closed the shift	Closed by the owner (forced) with a reason, expected minus ₦37,000, counted ₦40,000, payouts ₦39,000
Catering	Booked an order with a ₦500 cash deposit	One deposit with method "cash", not carried over

Screen	What was done	What the database shows
Pay a supplier	Paid ₦100 by bank transfer after choosing a method	One payment, method "bank_transfer", no payout, audit line "by bank transfer"

Not used by a person yet (no audit line exists for them): reversing a cash payout, a purchase or supplier payment marked "from the drawer", reversing a purchase or a supplier payment, customer credit payments and write-offs, catering payments and their reversal, "Correct the count", approving a stock count, voids and refunds, closing a shift as balanced.

8.3 What is not yet proved

- **Screens not used by a person:** the list at the end of 8.2d. The three correction screens (batches, payouts, price decisions) and most of the cash-out screens are now proved by use (8.2d). [S]
- **The calendar buttons** have not been tried on a real phone or in a real Outlook account. [S]
- **The new Pay a supplier screen** has been used once (a bank transfer). The "other" note rule, the cash-from-drawer choice and the greyed-out button have not been reported. [S]
- **Monnify automatic transfer confirmation** has not been tested end to end with real keys. [S]
- **Backups.** A restore into a scratch project has not been done. [S]
- **The offline Phase 0, dish price history and paper-entry screens** are not on the live site. Their database parts are applied and read, and have not been rehearsed (8.4, 8.5). [S]
- **The real stock-take, wastage and sale functions** were exercised by the live rehearsal for the sale, void, refund, wastage and stock-count paths. The batch path works for a dish that is not set to "made to order". [R]

8.4 Offline Phase 0 (save-once sales): evidence

This is evidence, not a new business rule. The rules are in 3.9.

- The database script `20261030_sale_once_a.sql` is applied. The owner's check returned 1, 1, 6, false, false, 0. On 4 October I confirmed by reading the live database that the six functions, the sale-code column and the unique rule exist and that 0 orders carry a code yet. [C]
- Device-side rules (connection states, draft privacy, paper reference, draft expiry, the 7-day purge) pass automated checks on the source branch. [L]
- **No live rehearsal of save-once sales has been done.** A rehearsal was written (`rehearsal_phase0_prices.sql`, covering this and 3.10) and **has not been run:** the owner declined the request to run it on 4 October. It is the next step before release. [S]
- Module D (UAT-OFF-01 to 15) in the Owner UAT script has not been run by a person on a real phone. The release gate in the Owner UAT script and the System Test Checklist applies: all 15 tests recorded, no duplicate order in double-submit, retry or timeout paths, no unsaved draft shown as saved, no customer data kept in cash drafts, health check and rehearsal clean, and a real Android phone in airplane mode. **Phase 0 must not be described as proved on the live site until Module D results exist.** [S]

8.5 Dish price history and paper entries: evidence

- **Price history.** Found applied on 4 October. I read from the live database: the history table with 8 rows (one starting price per current dish), the rule on recipes, the order-line rule, the lookup function, and the read rule limited to the business. 0 order lines carry a price row yet. [C]
- **Paper entries.** Applied 4 October (reported by the owner and confirmed by reading the live database): both tables, the three functions, the six order columns and the two guard rules; 0 entries. [C]

- **Not rehearsed, not used by a person.** Reading the SQL against the live data found defects D1 to D6 (Part 9). D1 is shown by a live read: for the dish "Eba & Egusi", whose current menu row is its second saved version, the lookup returns no price when given the menu row's own identifier and ₦1,500.00 when given the dish identifier. D2 is shown by the live rule on order status, which lists six values and not the one the approval function writes. D3 to D6 are read from the code. [C]

8.6 Checks made on 4 October 2026, and corrections to the first draft

- **Health check on the live database** (read-only): row security missing on 0 tables; visitor privileges **21** (expect 0); risky signed-in privileges **9** (expect 0); trigger functions visitors can run **2** (expect 0); privileged functions without a fixed path 0; write rules on money tables 0; lock triggers present; the eight reversal and adjustment functions present 8 of 8; shifts open 0; ingredients below zero stock 0; unread alerts 5. The three failing values come from the paper-entry script (R1). [C]
- **Totals read from the live database:** 52 tables, 1 view, 72 access rules (29 write rules), 114 application functions (38 trigger functions; 91 with extra privilege, all with a fixed path; 62 callable by signed-in people, 1 by visitors, a number-formatting helper), 56 triggers, 90 foreign keys. A further 188 functions in the public schema belong to the `btree_gist` extension, installed by the price history script and unused (R2). [C]
- **Corrections to the first draft of the Phase 0 text:** the first draft said the connection check runs every 20 seconds; the code checks every 25 seconds while online and at 5, 10, 20 and then 30 seconds after failures. It said the owner UAT script already held a Module E for paper entries and a late-entry test list in the UAT page; neither exists, so Module E was written for this version. It said the late-entry script was applied and verified; when I first read the live database on 4 October it was **not** applied, and the owner applied it later the same day. [C]

Part 9. Known gaps and open items

Ranked by what matters most. Each says what it is, what could go wrong, and the status.

Faults and gaps in the controls

Ref	What	Risk	Status
F0	Logging a batch fails for everyone. The trigger that stamps the recipe version on batches also tries to set a plate cost, which batches do not have. Error: <code>record "new" has no field "cost_per_plate_kobo"</code> . Confirmed live on 2 October 2026. No batch has been logged since 25 September.	Cooks cannot log batches. Stock is not reduced for batch dishes.	Closed 3 October 2026. Fix applied (<code>20261025_batch_trigger_fix.sql</code>) and a batch was logged in the rehearsal.
A1	Refund records (<code>order_adjustments</code>) can be written directly by cashiers and owners, and edited by owners.	A forged part refund lowers expected cash and can hide a drawer shortage.	Closed 3 October 2026. Direct writes refused, edits blocked.
A2	Audit lines can be added directly by any signed-in staff.	The audit trail can be padded or forged.	Closed 3 October 2026. Direct writes refused.
A3	Channel payouts can be written and edited directly by owners.	Payout figures can be altered after the check.	Closed 3 October 2026. Frozen; only <code>log_channel_payout</code> writes.
A4	Price decisions and batches can be written, edited or deleted directly by owners.	Decision history and batch history can be altered.	Closed 3 October 2026. Frozen; only <code>decide_price</code> and <code>log_batch</code> write. Corrections need a future function.
B1 to B3	Visitors still hold table privileges (blocked by row security), signed-in users hold TRUNCATE, TRIGGER and REFERENCES, and visitors can be given execute on trigger functions. None is reachable through the app.	Defence in depth only.	Closed 3 October 2026. Visitors hold no table privileges, signed-in users no longer hold TRUNCATE, TRIGGER or REFERENCES, and visitors cannot run any trigger function. Backups of the old grants are kept in two private tables.
B5	The sign-in screen needs to show who can sign in, so one server action (<code>list_staff</code>) returns the id, display name, role and active flag of a business's active staff to anyone who knows the business code , without signing in. Business codes are short names such as <code>demo-kitchen</code> . It never returns PINs, phones or emails.	An outsider can list staff names and roles of a business whose code they know, and then try PINs. Repeated wrong PINs lock the account (failed attempts and a lock time are saved, and one account lock is in the live audit trail). I have not tested the lockout limits.	Design choice. Needs a decision.

Ref	What	Risk	Status
R1	Table privileges re-opened by the paper-entry script (4 October 2026). <code>late_entries</code> and <code>late_entry_items</code> were created with the default grants: visitors hold every privilege on them, signed-in people hold every privilege including TRUNCATE, TRIGGER and REFERENCES, the view <code>dish_price_periods</code> is readable by visitors, and the guard function <code>late_entries_protect</code> can be run by visitors. Health check: visitor privileges 21 (expect 0), risky signed-in privileges 9 (expect 0), trigger functions visitors can run 2 (expect 0).	Not reachable through the app: row security is on and there is no write rule, and the guard rule refuses edits and deletes. But it undoes B1 to B3 and fails the health check.	Open. Fix script <code>20261102_hygiene_after_late_entries.sql</code> with check and rollback, tested locally, not applied .
R2	An unused extension was installed in the public schema. <code>20261031</code> installed <code>btree_gist</code> (188 functions in the public schema). Nothing uses it: no index of that kind exists and nothing depends on it.	Clutter and a larger surface; no known harm.	Open, low. Optional: <code>drop extension if exists btree_gist;</code> (in the fix script, commented out).
D1	Paper entries cannot be entered for any dish whose menu row is a later saved version. The screen and the database look up the price with the menu row's own identifier, but the history is kept by the dish identifier, which differs once a dish has been saved a second time. Live: "Eba & Egusi" (version 2) returns no price; 1 of 8 current dishes today.	The most-used test dish cannot be sold on paper: the screen shows "no price then" and the database refuses with "No valid price found". Every dish would be affected after its first edit.	Open. Fix before releasing the screens. In <code>submit_late_entry</code> and the screen, resolve the dish identifier from the menu row first.
D2	A paper entry paid by transfer cannot be approved. Approval writes the order status <code>transfer_pending</code> , but the orders table allows only draft, paid, cancelled, refunded, partially refunded and awaiting payment. The entry screen says "transfer stays pending until confirmed", but a split entry is posted as fully paid.	The approval of a pure-transfer entry fails with a database error. A split entry posts as paid even though its transfer part is unconfirmed.	Open. Fix before release. Owner decision: post transfers as paid, as manual transfers are today, or as "awaiting payment" with a payment request so something can confirm them.
D3	A paper entry's cash can be counted in the wrong shift, or twice. Expected cash counts an order by the moment it was recorded; a posted paper entry is recorded at approval. With "closed shift, already included" the cash also counts in whichever shift is open at approval; with "closed shift, late cash" it is added to the old shift as an adjustment and counts in the open shift.	The open shift can close short by the paper cash, or an old shift can show the cash twice. The screen tells the owner that cash is never moved to another shift, which is not what the database does.	Open. Fix before release. Needs a rule that excludes late entries from the open shift, or counts them by the real sale time. Owner decision needed.
D4	The order line records today's price row, not the price row of the sale time. The line's unit price is right (frozen on the entry), but the link to the price history is set by a rule that looks up the price at the moment the line is written.	The audit link points at the wrong price row when the price changed since the sale.	Open. Set the link from the entry's frozen price row.
D5	Cost of goods on a paper sale used today's ingredient prices.	Profit on late sales was estimated.	Closed on the database in version 1.4 (3.12): costed at the sale time, unknown prices held or labelled as estimates. Screens not released. [C]

Ref	What	Risk	Status
D6	The late-cash adjustment ignores the rules of the count adjustment function. A shift closed without a count cannot be adjusted, and an adjusted count cannot go below zero; the paper-entry approval inserts the adjustment directly and checks neither.	An adjustment can be added to a shift that has no count.	Open. Fix before release.
S1	The save-once functions return an existing sale before checking the caller's role. A signed-in cook or purchaser of the same business who already knows a sale code gets the sale's total, method, status and identifiers back.	Low: the code is a random identifier and there is no way to list codes. Out of line with every other function.	Open. Move the role check first. Confirm with a rehearsal.
S2	An approved cash payout cannot be undone from the screen. The database allows reversing it; the screen shows Reverse only on payouts recorded directly.	The owner must use "Correct the count" after the shift closes, or fix the screen.	Open, design choice.
S3	Supplier payments before 3 October 2026 have no method, and Part B is not applied.	Old payments show "method not recorded". A screen left open from before can still save a payment marked legacy.	Open. Apply Part B after the new screen is used once more.
S4	Batch and wastage costs are worked out in the browser. The batch screen sends the ingredient cost it calculated; the database stores it. Price history is not used to check it.	A tampered or stale screen could store a cost that differs from the price list. Batches are made now, so today's price is the right one; the gap is trust, not timing.	Open. Proposed: the database works the cost out and refuses a differing figure. Not built; needs the owner's go-ahead.
B4	<code>business_has_access</code> , <code>catering_enabled</code> , <code>payment_mode_of</code> and <code>trial_limits_apply</code> accept a business id from the caller. A signed-in person who guesses another business's id can learn whether it is active, has catering on, its payment mode, or is on a trial.	Low. No money or customer data.	Accepted and documented. These functions are needed by the access rules.

Cash-drawer accounting gaps

- **Closed in version 1.3:** catering deposits are now counted (a method is saved with them); cash purchases and supplier payments are taken out of expected cash when marked "paid from the cash drawer". [C][R][P]
- **A shift closed by an owner without a count** has no difference and cannot be adjusted.
- **A purchase or supplier payment that is NOT marked "paid from the cash drawer"** is not taken out of expected cash by design, because it was paid from somewhere else. If a person forgets the box, the drawer shows an apparent overage.
- **Reversals after a shift has closed.** A payout reversed with its purchase or payment after the shift closed leaves the shift figure as it was and writes an audit line ("cash payout left on closed shift"). The owner uses "Correct the count" if needed. [C][R]
- **Paper entries and shifts** (D3).

Other open items

- **Correction limits.** The six batches logged before 3 October 2026 cannot be reversed (no stock trail). A payout's mismatch alert is not linked to the payout and must be dismissed by hand. A dish price edited by

hand is not timestamped, so the price guard on reversing a decision compares prices only.

- **Stale shift.** The 25 September shift in Demo Kitchen was closed by the owner on 3 October 2026. The shift opened in the 3 October test was also closed by the owner the same day; the database shows no open shifts. [C]
- **Test data in the Demo Kitchen.** The 3 October shift holds ₦39,000 of test payouts (including an unreversed ₦20,000 test entry) and a forced close. They stay as test records.
- **Release state.** The live site is `main` at the supplier-method release. The offline, price-history and paper-entry code is in the source branch only. Releasing it would put the paper-entry screens on the live site with defects D1 to D4 and D6 in them, and would also need the fix for R1 applied. [C]
- **PIN hashing.** The upgrade (a stronger scheme with a secret pepper and re-hashing at sign-in) is planned and not built.
- **Public prices for visitors.** Prices are shown to signed-in people only. Showing them to signed-out visitors needs a decision.
- **Expiry reminder emails** exist but are switched off and not scheduled.
- **Backups.** Database backups and point-in-time recovery should be confirmed in the Supabase dashboard. This document cannot confirm them.
- **No staging environment.** Changes are tested locally and then run on the live database.
- **Older sales are costed at today's prices.** Sold lines from before 1 October 2026 have no frozen cost (Part 4.5). Sales since then are frozen.
- **Other deletes left open by decision:** suppliers, recipes, wastage entries, unit conversions and alerts can still be edited or deleted by owners. Each is a configuration or log table, not a ledger.
- **Recipe price edits.** An owner can change a dish's price directly without a price decision record. Sales keep the price they were sold at. Since 4 October 2026 such an edit also adds a row to the dish's price history (3.10); the rule that does this is on the live database.

Part 10. How the ledger programme got here

Step	What	Released	SQL files
Till price path	Cash and split sales saved by the database, which reads menu prices	PR #36	20261015
Locks	Catering table lock, sales delete lock, ledger delete lock	2 Oct	20261016 to 20261018
1	Catering payment ledger	PR #37	20261019
2	Supplier ledger	PR #38	20261020
3	Customer credit ledger	PR #39	20261021
4	Purchase ledger and reversals	PR #40	20261022
4b	Ingredient stock, price, unit and delete lock	PR #41	20261023
5	Closed cash drawers, adjustments, force close	PR #42	20261024
6	Security and safety sweep: batch fix, permission clean-up, locks on refund records, audit trail, payouts, price decisions and batches	PR #43 (SQL applied live 3 Oct)	20261025, 20261026
7	Owner corrections for payouts, price decisions and batches	PR #44 (SQL applied live 3 Oct)	20261027
8	Cash paid out of the drawer; catering deposit method	PR #47 (released 3 Oct; part A and part B applied live 3 Oct)	20261028_cash_out_a / b
8b	Calendar entries for catering orders (browser only, no SQL)	PR #48 (released 3 Oct)	none
8c	Supplier payment method	PR #49 (released 3 Oct; part A applied; part B not applied)	20261029_supplier_method_a / b
P0	Offline Phase 0: save-once sales	Source branch only (SQL applied 4 Oct)	20261030_sale_once_a
P1a	Dish price history	Source branch only (SQL applied)	20261031_dish_prices_a
P1b	Paper (late) entries	Source branch only (SQL applied 4 Oct)	20261101_late_entries_a
fix	Hygiene after the paper-entry script	Proposed, not applied	20261102_hygiene_after_late_entries

Each step shipped as: database change, a check query, a rollback, a tested local run, and a verified release.

Part 11. Sign-off checklist for the security and safety sweep

#	Check	Result on 4 October 2026
1	Every table has row security	Done: 52 of 52 [C]
2	No rule opens any table to signed-out visitors	Done: 0 rules [C]
3	Every privileged function has a fixed search path	Done: 91 of 91 [C]
4	Every function that moves money checks role and business	Done: every writer function listed in Part 6 checks the role in its body and filters by the caller's business [C][R]
5	Money tables cannot be written directly	Done for sales, catering, credit, supplier, purchases, drawers, cash payouts, stock, refund records, the audit trail, channel payouts, price decisions, batches and dish prices [C][R]. Paper entries: row security blocks writes, but the privileges are open (R1)
6	Finished records cannot be edited or deleted	Done for all ledgers, cash payouts, refund records, the audit trail, channel payouts, price decisions, batches and started dish prices [C][R]. Paper entries are guarded by a rule, not yet rehearsed [C]
7	Each business sees only its own data	Done: 6 table reads and 4 actions tested against another business [R]
8	Staff secrets cannot be read or written from the browser	Done [C][R]
9	Every kind of money event is accounted for	Done: Part 6 lists 41 (events 40 and 41 are not yet used by the live site)
10	Every core path works end to end, including the three corrections	Done at database level: sale, void, refund, purchase, reversal, payments, stock count, shift, batch and price decision all ran in the rehearsal [R]
11	New screens used by a person in Demo Kitchen	Partly done [P]: the three correction screens, the cash-out screens (limit, request, approve, decline, owner payout, forced close), the catering deposit method and one supplier payment by transfer. Not done : the list at the end of 8.2d, the calendar buttons on a phone, and all offline, price history and paper-entry screens
12	Backups confirmed	Not done : needs the Supabase dashboard
13	The health check returns the expected values	Fails since 4 October: 3 of the first 8 values (R1). Passes after the fix script is applied (tested locally)

#	Check	Result on 4 October 2026
14	The self-undoing rehearsal is clean after the latest changes	Clean at 200 steps on 3 October. Not run since the 4 October SQL; a rehearsal for save-once and price history is written and not run

Sign-off is not yet possible. The database controls are in place for everything on the live site, but four items are outstanding: the remaining screen walk-through (check 11), confirmation of backups (check 12), the permission fix (check 13) and a rehearsal after the 4 October SQL (check 14). The paper-entry defects (D1 to D6) must be fixed before those screens are released.

Technical appendices

All appendices were read from the live database and the code on 2, 3 and 4 October 2026 unless stated. They are for engineers and auditors.

Totals on 4 October 2026: 52 tables (50 app tables and 2 private grant-backup tables) and 1 view; 72 access rules (29 of them write rules, none on the money tables); 114 application functions (76 that are not triggers, 38 trigger functions, 91 with extra privilege, all with a fixed search path); a further 188 functions in the public schema that belong to the unused `btree_gist` extension (R2); 56 triggers; 90 foreign keys. The 5 scheduled jobs and 2 vault secrets were read on 2 and 3 October and have not changed.

Appendix A. Tables and who can write to them

"Direct" means a signed-in person writing to the table from the browser. **A dash means the browser cannot** (writes go through functions or the server). "Any member" means any signed-in person of the business. Read access is always limited to the person's own business.

Table	Kind	Read	Direct insert	Direct update	Direct delete
archived_app_state_snapshots	Archived	nobody	-	-	-
archived_saved_recipe_configs	Archived	nobody	-	-	-
audit_logs	Audit trail	owner, supa_admin	-	-	-
batches	Production log	cook, owner, supa_admin	-	-	-
business_features	Settings	any member	-	-	-
business_news_prefs	Settings	any member	-	-	-
business_payment_settings	Settings	owner, cashier, supa_admin	-	-	-
businesses	Tenant record	own business (members); all (platform admin)	owner, supa_admin (own row)	owner, supa_admin (own row; billing and status columns guarded by a trigger); platform admin	owner, supa_admin (refused while ledger records exist)
cash_drawer_adjustments	Ledger	owner, cashier, supa_admin	-	-	-
cash_drawer_payments	Ledger	owner, cashier, supa_admin	-	-	-
cash_drawers	Ledger	owner, cashier, supa_admin	-	-	-
catering_deposits	Catering order	owner, cashier, supa_admin	-	-	-
catering_order_items	Catering order	cook, owner, cashier, purchaser, supa_admin	-	-	-

Table	Kind	Read	Direct insert	Direct update	Direct delete
catering_payments	Ledger	owner, cashier, supa_admin	-	-	-
catering_reminder_log	System log	nobody (server only)	-	-	-
channel_payouts	Payout record	owner, supa_admin	-	-	-
contact_messages	Platform	platform admin	- (server)	platform admin	-
credit_payments	Ledger	owner, cashier, supa_admin	-	-	-
customer_credits	Ledger	owner, cashier, supa_admin	-	-	-
daily_summary_log	System log	nobody (server only)	-	-	-
dish_prices	Price history	any member (own business, active plan)	-	-	-
drawer_settings	Settings	owner, cashier, supa_admin	-	-	-
ingredient_grade_prices	Price record	cook, owner, cashier, purchaser, supa_admin	-	-	-
ingredients	Stock and price	cook, owner, cashier, purchaser, supa_admin	owner, purchaser, supa_admin (stock and price columns must start at zero)	owner, purchaser, supa_admin (stock, cost, grade, season, price date locked; unit locked once history exists)	owner, supa_admin (refused once history exists)
late_entries	Paper entries	cashier, owner, supa_admin	- (no row-security rule; privilege open, R1)	- (same)	- (same)
late_entry_items	Paper entries	cashier, owner, supa_admin	- (no row-security rule; privilege open, R1)	- (same)	- (same)
margin_flags	Alerts	by the role the alert is for	owner, supa_admin	by role (to mark seen)	owner, supa_admin
news_feed_status	System	nobody	-	-	-
news_items	Reference	cook, owner, cashier, purchaser, supa_admin	-	-	-
order_adjustments	Sales record (voids, refunds)	owner, cashier, supa_admin	-	-	-

Table	Kind	Read	Direct insert	Direct update	Direct delete
order_items	Sales	owner, cashier, supa_admin	-	-	-
orders	Sales	owner, cashier, supa_admin	-	-	-
payment_events	System	nobody	-	-	-
payment_requests	Payments	owner, cashier, supa_admin	-	-	-
platform_settings	Platform	nobody directly (read through public_settings)	-	-	-
price_decisions	Decision log	owner, supa_admin	-	-	-
purchases	Ledger	owner, purchaser, supa_admin	-	-	-
recipe_items	Configuration	cook, owner, cashier, purchaser, supa_admin	cook, owner, supa_admin	cook, owner, supa_admin	owner, supa_admin
recipe_variants	Configuration	owner, supa_admin	-	-	-
recipes	Configuration	cook, owner, cashier, purchaser, supa_admin	cook, owner, supa_admin	cook, owner, supa_admin	owner, supa_admin
staff_users	Staff	owner, supa_admin (8 non-secret columns only)	no privilege	no privilege	-
stock_count_lines	Stock	cook, owner, purchaser, supa_admin	-	-	-
stock_counts	Stock	cook, owner, purchaser, supa_admin	-	-	-
stock_movements	Ledger	owner, purchaser, supa_admin	-	-	-
subscription_payments	Platform	nobody (server only)	-	-	-
subscription_remitter_log	System log	nobody (server only)	-	-	-

Table	Kind	Read	Direct insert	Direct update	Direct delete
supplier_transactions	Ledger	owner, purchaser, supa_admin	-	-	-
suppliers	Configuration	owner, purchaser, supa_admin	owner, purchaser, supa_admin	owner, purchaser, supa_admin	owner, supa_admin
unit_conversions	Configuration	cook, owner, cashier, purchaser, supa_admin	owner, purchaser, supa_admin	owner, purchaser, supa_admin	owner, supa_admin
wastage_logs	Log	cook, owner, purchaser, supa_admin	cook, owner, purchaser, supa_admin	owner, supa_admin	owner, supa_admin (stock is put back)

Notes:

- `staff_users` has access rules that look as if owners can insert and update, but signed-in users hold **no** insert or update privilege on any column, so those rules have no effect. PIN hash and salt are not readable. [R]
- Ledger tables also have a guard trigger that refuses any change or deletion by a signed-in person (Appendix C), so a mistaken access rule added later would still be refused.
- `dish_price_periods` is a view over `dish_prices` that shows each price with its end time. It runs with the caller's rights, so row security applies. Visitors currently hold a read privilege on it (R1). [C]
- `late_entries` and `late_entry_items` have row security on and only a read rule. Signed-in people and visitors nevertheless hold table privileges on them (R1), which the fix script removes. [C]

Appendix B. Functions callable from the app

Every function below fixes its search path. "Definer" functions run with the privileges of their owner and therefore must check the caller themselves; each lists the roles it accepts.

Function	Runs as	Roles accepted	Purpose
<code>open_cash_drawer(float)</code>	definer	owner, cashier, supa_admin	Open a shift (one per business)
<code>adjust_closed_drawer(drawer, amount, reason)</code>	definer	owner, supa_admin	Add a count adjustment to a closed shift
<code>create_cash_order(channel, tier, method, cash, transfer, items)</code>	definer	owner, cashier, supa_admin	Cash, transfer or split sale
<code>create_transfer_order(channel, tier, items)</code>	definer	owner, cashier, supa_admin	Transfer sale
<code>create_credit_order(channel, tier, customer, phone, items)</code>	definer	owner, cashier, supa_admin	Credit sale and debt
<code>cancel_unpaid_order(order, reason)</code>	definer	owner, cashier, supa_admin	Cancel an order that was never paid
<code>adjust_order(order, type, reason, amount)</code>	definer	owner, cashier, supa_admin	Void or refund (cashier: own sales only)
<code>record_credit_payment(credit, amount, method)</code>	definer	owner, cashier, supa_admin	Debt payment
<code>write_off_credit(credit, amount, reason)</code>	definer	owner, supa_admin	Write off
<code>reverse_credit_entry(entry, reason)</code>	definer	owner, supa_admin	Reverse a debt entry
<code>create_manual_credit(customer, phone, amount, note)</code>	definer	owner, supa_admin	Debt by hand
<code>create_catering_order(...)</code>	definer	owner, cashier, supa_admin	Catering order with deposit and deposit method (12 arguments; the older 11-argument version was removed in part B on 3 October)
<code>record_catering_payment_v2(order, amount, method)</code>	definer	owner, cashier, supa_admin	Catering payment
<code>record_catering_payment(order, amount)</code>	definer	(hands off to v2)	Older name, same checks
<code>reverse_catering_payment(payment, reason)</code>	definer	owner, supa_admin	Reverse a catering entry
<code>set_catering_status(order, status)</code>	definer	owner, cashier, supa_admin	Change catering status

Function	Runs as	Roles accepted	Purpose
<code>log_purchase(...)</code>	defin er	owner, purchaser, supa_admin	Log a purchase with snapshot
<code>reverse_purchase(purchase, reason)</code>	defin er	owner, supa_admin	Reverse the latest purchase
<code>reverse_payout(payout, reason)</code>	defin er	owner, supa_admin	Reverse a channel payout
<code>reverse_price_decision(decision, reason)</code>	defin er	owner, supa_admin	Reverse a price decision; restores the dish price for a published decision
<code>reverse_batch(batch, reason)</code>	defin er	owner, supa_admin	Reverse a batch; puts the stock back
<code>set_ingredient_price(ingredient, price, grade, season)</code>	defin er	owner, purchaser, supa_admin	Change a price
<code>record_supplier_payment(supplier, amount, note)</code>	defin er	owner, purchaser, supa_admin	Supplier payment, older version: saves the payment with method "legacy". To be removed in part B
<code>reverse_supplier_payment(payment, reason)</code>	defin er	owner, supa_admin	Reverse a supplier payment
<code>log_batch(...)</code>	defin er	cook, owner, supa_admin	Log a batch
<code>submit_stock_count(note, opening, lines)</code>	defin er	cook, owner, purchaser, supa_admin	Submit a count
<code>decide_stock_count(count, approve)</code>	defin er	owner, supa_admin	Approve or reject a count
<code>log_channel_payout(...)</code>	defin er	owner, supa_admin	Record a channel payout and compare
<code>decide_price(recipe, decision, price, by)</code>	defin er	owner, supa_admin	Record a price decision
<code>save_recipe_version(...)</code>	invoker	owner, supa_admin	Save a new recipe version (row security applies)
<code>save_recipe_variant(...), delete_recipe_variant(variant)</code>	defin er	owner, supa_admin	Recipe variants
<code>set_news_enabled(enabled)</code>	defin er	owner, supa_admin	News watch switch
<code>set_payment_mode(mode)</code>	defin er	owner, supa_admin	Manual or automatic transfer confirmation
<code>ingredient_ids_with_history()</code>	defin er	owner, purchaser, supa_admin	Ingredients whose unit is locked

Function	Runs as	Roles accepted	Purpose
<code>ingredient_has_history(id)</code>	defin er	helper	Used by the ingredient guard; own business only
<code>list_businesses_for_review(status)</code>	defin er	platform admin (checked by role)	Business approvals
<code>public_settings()</code>	defin er	any signed-in person	Prices, locked-screen text, expiry banner
<code>business_has_access(id), catering_enabled(id), payment_mode_of(id), trial_limits_apply(id)</code>	defin er	helpers (B4)	Used inside access rules and screens
<code>audit_naira(kobo)</code>	invok er	helper (visitors can run it)	Formats a naira amount
<code>record_cash_payout(amount, category, note)</code>	defin er	owner, cashier, supa_admin	Take cash out of the open shift, or a request when a cashier is over the limit
<code>approve_cash_payout(request)</code>	defin er	owner, supa_admin	Approve a waiting request
<code>decline_cash_payout(request, reason)</code>	defin er	owner, supa_admin	Decline a waiting request
<code>reverse_cash_payout(payout, reason)</code>	defin er	owner, supa_admin	Reverse a payout while its shift is open
<code>set_drawer_payout_limit(limit)</code>	defin er	owner, supa_admin	The cashier limit for one shift
<code>log_purchase_from_drawer(...)</code>	defin er	owner, purchaser, supa_admin	A cash purchase and its cash payout, saved together
<code>record_supplier_payment_v2(supplier, amount, note, method)</code>	defin er	owner, purchaser, supa_admin	Supplier payment with its method (the new screen calls this)
<code>record_supplier_payment_from_drawer(supplier, amount, note)</code>	defin er	owner, purchaser, supa_admin	Supplier payment and its cash payout, saved together (v2 calls it for "cash from the drawer")
<code>create_cash_order_once(code, ...), create_credit_order_once(code, ...), create_transfer_order_once(code, ...)</code>	defin er	owner, cashier, supa_admin for a new sale (see S1)	The sale functions with a sale code: save once
<code>find_sale_by_client_id(code)</code>	defin er	owner, cashier, supa_admin	Was a sale with this code saved? Read-only
<code>dish_price_at(dish, time)</code>	invok er	any signed-in person (row security applies)	The price that applied to a dish at a moment

Function	Runs as	Roles accepted	Purpose
<code>set_dish_price(dish, price, from)</code>	defin er	owner, supa_admin	Set a dish price now or schedule one
<code>cancel_dish_price(price)</code>	defin er	owner, supa_admin	Cancel a scheduled price before it starts
<code>refresh_dish_prices()</code>	defin er	any signed-in person (own business)	Bring started scheduled prices up to date
<code>submit_late_entry(...)</code>	defin er	owner, cashier, supa_admin	Submit a paper (late) entry
<code>reject_late_entry(entry, reason)</code>	defin er	owner, supa_admin	Reject a paper entry
<code>approve_and_post_late_entry(entry, resolution, notes)</code>	defin er	owner, supa_admin	Approve a paper entry and post it as a sale

Server-only functions (no signed-in person can call them): `apply_stock_count`, `attach_payment_account`, `raise_catering_alerts`, `read_payment_connection`, `recipe_plate_cost_kobo`, `record_provider_payment`, `save_payment_connection`, `signup_business`, `supplier_balance_kobo`, `to_base_qty`, and (added 3 and 4 October) `apply_due_dish_prices`, `sale_once_existing`, `sale_once_stamp`, `record_supplier_payment_core`.

Appendix C. Triggers

Table	Trigger	Function	When	What it does
batches	batches_stamp_version	stamp_batch_version	before insert	Stamps the recipe version only (own function since 3 October 2026, FO fix)
batches	batches_no_direct_insert	block_direct_insert	before insert	Refuses direct inserts from signed-in people (A4)
batches	batches_no_change	ledger_block_change	before update, delete	Refuses edits and deletes (A4)
batches	batches_stock_mode_check	check_batch_stock_mode	before insert	Refuses batches for "made to order" dishes; labels stock changes "batch use". Skips both for a reversal row
businesses	audit_business_review	audit_business_review	after update	Audit line for status changes
businesses	businesses_status_guard	businesses_status_guard	before insert, update	Only the platform changes status; billing columns only through the server
cash_drawer_adjustments	cash_drawer_adjustments_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
cash_drawer_payouts	cash_drawer_payouts_no_direct_insert	block_direct_insert	before insert	Refuses direct inserts: "Cash can only be taken out of the drawer from the Cash drawer screen."
cash_drawer_payouts	cash_drawer_payouts_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
cash_drawers	cash_drawers_protect	cash_drawers_protect	before insert, update, delete	Refuses direct open, close, edit, delete by signed-in people
cash_drawers	cash_drawers_pending_guard	cash_drawers_pending_guard	before update	Refuses closing a shift while a payout request is waiting
catering_payments	catering_payments_check_reversal	catering_payments_check_reversal	before insert	Reversal rules
catering_payments	catering_payments_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
catering_payments	catering_payments_recompute	catering_payments_recompute	after insert	Recomputes the order's cached totals
credit_payments	credit_payments_check	credit_payments_check	before insert	Balance, owner-only and reversal rules
credit_payments	credit_payments_no_change	ledger_block_change	before update, delete	Refuses edits and deletes

Table	Trigger	Function	When	What it does
credit_payments	credit_payments_recompute	credit_payments_recompute	after insert	Recomputes paid, written off, settled
customer_credits	customer_credits_protect	customer_credits_protect	before update	Refuses any direct change by a signed-in person
dish_prices	dish_prices_protect	dish_prices_protect	before insert, update, delete	Refuses direct inserts; started rows frozen; only a scheduled price can be cancelled; never deleted
ingredients	ingredients_log_stock	log_stock_movement	after update	Writes a stock movement with its reason
ingredients	ingredients_protect	ingredients_protect	before insert, update	Locks stock, price, grade, season, price date; locks unit once history exists
ingredients	ingredients_protect_delete	ingredients_protect	before delete	Refuses delete once history exists
ingredients	ingredients_trial_limit	enforce_trial_ingredient_limit	before insert	Trial plan limit
ingredients	trg_audit_ingredient_cost	audit_ingredient_cost	after update	"Cost changed" audit line
late_entries	late_entries_protect	late_entries_protect	before update, delete	Refuses edits and deletes except by the three paper-entry functions and the server
late_entry_items	late_entry_items_protect	late_entries_protect	before update, delete	Same guard for the entry lines
order_adjustments	order_adjustments_no_direct_insert	block_direct_insert	before insert	Refuses direct inserts (A1)
order_adjustments	order_adjustments_no_change	ledger_block_change	before update, delete	Refuses edits and deletes (A1)
order_adjustments	trg_audit_order_adjustment	audit_order_adjustment	after insert	"Order adjusted" audit line
order_items	order_items_apply_stock	apply_sale_stock	after insert	Reduces stock for "made to order" dishes
order_items	order_items_lock_cost	order_items_lock_cost	before update	Keeps the plate cost fixed
order_items	order_items_stamp_version	stamp_recipe_version	before insert	Stamps recipe version and plate cost
order_items	order_items_set_dish_price	order_items_set_dish_price	before insert	Records which dish price row the line used (D4)
orders	orders_payment_guard	orders_payment_guard	before insert, update	Rules for who can mark an order paid
orders	orders_reverse_stock	reverse_sale_stock	after update	Puts stock back when a sale is voided
price_decisions	price_decisions_no_direct_insert	block_direct_insert	before insert	Refuses direct inserts (A4)

Table	Trigger	Function	When	What it does
price_decisions	price_decisions_protect	price_decisions_protect	before update, delete	Refuses edits and deletes, except the database emptying the link to a deleted recipe or user (A4)
price_decisions	trg_audit_price_published	audit_price_published	after insert	"Price published" audit line
purchases	purchases_check	purchases_check	before insert	Reversal rules; refuses direct inserts
purchases	purchases_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
purchases	purchases_cash_payout_follow	cash_payout_follow_reversal	after insert (reversals only)	Reverses the linked cash payout when a purchase is reversed
recipe_items	recipe_items_trial_limit	enforce_trial_recipe_items_limit	before insert	Trial plan limit
recipes	recipes_cleanup_variants	cleanup_recipe_variants	after delete	Removes variants of a deleted recipe
recipes	recipes_set_dish_id	set_recipe_dish_id	before insert	Groups versions of one dish
recipes	recipes_trial_limit	enforce_trial_recipe_limit	before insert	Trial plan limit
recipes	recipes_record_price	recipes_record_price	after insert, update of price or current version	Adds a price history row whenever the price changes by any route
stock_movements	stock_movements_alerts	raise_stock_alerts	after insert	Low-stock and below-zero alerts
supplier_transactions	supplier_transactions_check_reversal	supplier_transactions_check_reversal	before insert	Reversal rules
supplier_transactions	supplier_transactions_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
supplier_transactions	supplier_transactions_cash_payout_follow	cash_payout_follow_reversal	after insert (reversals only)	Reverses the linked cash payout when a supplier payment is reversed
wastage_logs	wastage_logs_apply_stock	apply_wastage_stock	after insert	Reduces stock
wastage_logs	wastage_logs_restore_stock	apply_wastage_stock	after delete	Puts stock back

The shared guard `ledger_block_change` refuses any change or deletion when a signed-in person is acting. The server key and the SQL editor pass, for administration only.

Appendix D. Ledger rules at a glance

Ledger	Entry kinds	Amount	Reversal	Reason	Totals kept on the parent
catering_payments	deposit, payment, reversal	deposit and payment above zero; reversal below zero	One per entry, same order, exact amount, owner only	5+ chars on reversal	deposit and additional payments, recomputed
credit_payments	payment, write_off, reversal	payment and write-off above zero; reversal below zero	One per entry, same debt, exact amount, owner only	5+ chars on write-off and reversal	paid, written off, settled, recomputed
supplier_transactions	purchase_on_credit, payment, reversal, purchase_reversal	above zero	Payment reversal and credit-purchase reversal, once each, owner only	5+ chars on both reversals	none: balance worked out from entries
purchases	purchase, reversal	purchase above zero; reversal below zero (quantity and amount)	One per purchase, same ingredient, exact quantity and amount, owner only	5+ chars	stock and price on the ingredient
cash_drawers	one row per shift	float, count, expected, difference	not applicable	5+ chars when closed by an owner	breakdown saved at close
cash_drawer_adjustments	adjustment	not zero	An adjustment is corrected by another adjustment	5+ chars	adjusted count worked out
channel_payouts	entry, reversal	reversal carries negated figures	One per entry, owner only (reverse_payout)	5+ chars	none: totals sum the rows
price_decisions	entry, reversal	reversal swaps previous and suggested price	One per entry, owner only; price restored only if unchanged and no later published decision stands	5+ chars	dish price
batches	entry, reversal	reversal carries negated yield and costs	One per entry, owner only; full stock put back; needs the batch's stock trail	5+ chars	stock on the ingredients
stock_movements	one per stock change	signed change and balance after	not applicable	reason code	balance on the ingredient
cash_drawer_payouts	payout, request, decline, reversal	payout, request and decline above zero; reversal below zero	A payout is reversed once (owner, shift open); a request is approved or declined once (owner). One settling entry per request or payout	5+ chars note on every entry; 5+ chars reason on decline and reversal	the closed shift keeps a paid-out figure

Ledger	Entry kinds	Amount	Reversal	Reason	Totals kept on the parent
dish_prices	one row per price (starting price, owner, recipe change)	above zero	Not reversed: a started price is frozen; a scheduled price can be cancelled once before it starts	none	the menu price on recipes is a copy of today's price
late_entries	submitted, needs shift review, rejected, posted	total above zero; cash plus transfer equal the total	Not reversed: rejected by an owner or posted as a sale	3+ chars outage reason; 5+ chars to reject	none

Appendix E. Server routes

All 17 routes run on the server with the service key. Each one checks the caller before doing anything. (`ping` is new on 4 October and exists only in the source branch; the other 16 are on the live site.)

Route	Purpose	Who calls it	How the caller is checked
business-signup	New business sign-up	Visitor	Public by design; PIN hashed on the server; business starts "pending"
staff-pin-login	PIN sign-in and the staff list for the sign-in screen	Visitor	Checks the PIN hash; failed attempts lock the account. The staff list needs no sign-in (B5)
staff-admin	Staff, roles, PINs	Owner, Supa Admin	Login token; business and role from the token only
cash-drawer-close	Close your own shift	Cashier, owner	Login token
cash-drawer-force-close	Owner closes a shift left open	Owner, Supa Admin	Login token
payment-connect	Connect Monnify; keys go to the vault	Owner	Login token
payment-start	Ask Monnify for a transfer account	Till	Login token
payment-test	Check saved Monnify keys	Owner	Login token
payment-webhook	Monnify tells NairaPlate money arrived	Monnify	Signature made with the shop's own secret key
platform-admin	Support console	Platform admin	Login token; role from the token only
contact	Website enquiry forwarded to the platform inbox	Visitor	Public by design; payload size limited
daily-summary	Evening summary email	Scheduler	Bearer secret
catering-reminders	Catering reminder emails	Scheduler	Bearer secret
expiry-reminders	Plan expiry emails (off until an admin switches them on)	Scheduler	Bearer secret
news-watch	Reads news feeds for ingredient price headlines	Scheduler	Bearer secret
ping	Connection check for the Till: answers with nothing; no database work	Visitor (the Till)	Public by design; read-only; no data in or out
summary-unsubscribe	"Stop these emails" link	Owner (from email)	Link signed with the summary secret; two steps (view, then confirm)

Scheduled jobs (database scheduler): daily summary 19:00 UTC daily; news watch at minute 20 every hour; catering alerts, catering morning email 05:30 UTC daily; catering evening email 17:30 UTC daily. **Vault secrets:**

the site address and the scheduler secret. Expiry reminders are not scheduled.

Appendix F. Audit events

Seen in the live audit trail to 3 October 2026 (153 lines, 29 types; the cash payout events, drawer_limit_changed, drawer_force_closed, batch_reversed, payout_reversed and price_decision_reversed are new since 2 October): login_success 87, login_failed 17, business_created 5, pin_reset 4, cash_payout_recorded 3, cash_payout_requested 3, cost_changed 3, business_approved 3, cash_payout_declined 2, drawer_limit_changed 2, drawer_discrepancy 2, role_changed 2, business_rejected 2, price_published 2, stock_count_submitted 2, and one each of account_locked, batch_reversed, cash_payout_approved, catering_order_created, contact_message_handled, drawer_force_closed, drawer_opened, email_undelivered, payout_reversed, platform_setting_changed, price_decision_reversed, security_alert_undelivered, staff_created, supplier_payment_recorded. **Earlier count (to 2 October 2026):** login_success 76, login_failed 15, business_created 5, business_approved 3, cost_changed 3, pin_reset 2, role_changed 2, business_rejected 2, price_published 2, stock_count_submitted 2, security_alert_undelivered 1, account_locked 1, email_undelivered 1, platform_setting_changed 1, contact_message_handled 1, staff_created 1.

Written by database functions and triggers: cash_payout_approved, cash_payout_declined, cash_payout_left_on_closed_shift, cash_payout_recorded, cash_payout_requested, cash_payout_reversed, dish_price_cancelled, dish_price_scheduled, dish_price_started, drawer_limit_changed, late_entry_posted, late_entry_rejected, late_entry_submitted, business_approved, business_created, business_reactivated, business_rejected, business_suspended, catering_order_created, catering_payment_recorded, catering_payment_reversed, cost_changed, credit_created_manual, credit_entry_reversed, credit_payment_recorded, credit_written_off, drawer_count_adjusted, drawer_opened, order_adjusted, payment_mode_changed, price_published, purchase_reversed, payout_reversed, price_decision_reversed, batch_reversed, stock_count_approved, stock_count_rejected, stock_count_submitted, supplier_payment_recorded, supplier_payment_reversed.

Written by server routes: login_success, login_failed, account_locked, staff_created, role_changed, staff_deactivated, pin_reset, drawer_discrepancy, drawer_force_closed, platform_unlock_staff, emergency_owner_pin_reset, emergency_reset_blocked, security_alert_undelivered, email_undelivered, business_approved, business_rejected, business_suspended, business_reactivated, contact_message_handled, subscription_payment_recorded, platform_setting_changed, payment_provider_connected, feature_switched.

Also written by the database function `cancel_unpaid_order` : unpaid_order_cancelled (missing from version 1.0 of this list). Added in version 1.2: payout_reversed, price_decision_reversed and batch_reversed (proved by the rehearsal). Added in version 1.3: the cash payout events, drawer_limit_changed, the three dish price events and the three paper-entry events. The dish price and paper-entry events have not been written on the live database and are not yet rehearsed.

Event types that no one has triggered yet in the live data (for example credit_payment_recorded, drawer_opened, purchase_reversed) are proved by the rehearsals on 2 and 3 October 2026, where each one was written and then undone. [R]

Appendix G. SQL files for the programme

All files are in `supabase/external/`. Each step has the change, a check query and a rollback. Run order matters: a "part A" before its screen is released, a "part B" after.

Files	What
20261015_till_price_a / b	Till price from the menu; direct sale writes closed
20261015_drop_unused_tables, archive_unused_tables	Unused tables removed or archived
20261016 to 20261018	Catering table lock, sales delete lock, ledger delete lock
20261019_catering_payments	Step 1
20261020_supplier_ledger_a / b	Step 2
20261021_credit_ledger_a / b	Step 3
20261022_purchase_ledger_a / b	Step 4
20261023_ingredient_guard	Step 4b
20261024_cash_drawer_ledger_a / b	Step 5
20261025_batch_trigger_fix	F0 fix. Applied 3 October 2026
20261025_hygiene_batch	B1 to B3. Applied 3 October 2026
20261026_a1_refund_records_lock	A1. Applied 3 October 2026
20261026_a2_audit_trail_lock	A2. Applied 3 October 2026
20261026_a3_channel_payouts_lock	A3. Applied 3 October 2026
20261026_a4_decisions_batches_lock	A4. Applied 3 October 2026
20261027_owner_corrections_a	Step 7: owner corrections (one script, no part B). Applied 3 October 2026
20261028_cash_out_a / b	Step 8: cash paid out of the drawer (part A, applied 3 Oct); catering deposit method (part B, applied 3 Oct)
20261029_supplier_method_a / b	Supplier payment method. Part A applied 3 Oct. Part B not applied
20261030_sale_once_a	Offline Phase 0: save-once sales. Applied 4 Oct
20261031_dish_prices_a	Dish price history. Applied (found applied on 4 Oct)
20261101_late_entries_a	Paper (late) entries. Applied 4 Oct. Has defects D1 to D4 and D6, and caused R1
20261102_hygiene_after_late_entries	Proposed, not applied. Fixes R1 (an optional line for R2)
system_health_check.sql	Read-only health check of the locks and permissions. Run 4 Oct: fails 3 of the first 8 values (R1)
20261103_ingredient_prices_a	Ingredient price history. Applied; check read live 5 Oct, matches

Files	What
20261105_late_entry_cost_at	Paper-sale food cost at the sale time. Applied; check read live 5 Oct, matches (not rehearsed)
rehearsal_phase0_prices.sql	Self-undoing rehearsal for save-once sales and dish price history. Written, not run
rehearsal_demo_kitchen.sql	The self-undoing rehearsal used in Part 8 (200 steps on 3 Oct)

Appendix H. Glossary

Word	Meaning
Kobo	One hundredth of a naira. All money is stored in whole kobo.
Ledger	A record where lines are only added, never changed.
Reversal	A new line that cancels an earlier line without erasing it.
Adjustment	A new line that corrects a closed shift's count without changing it.
Row security	The database rule that decides which rows a person can see or change.
Security definer	A database function that runs with extra privilege, so it must check who is calling.
Trigger	A database rule that runs automatically when a row is added, changed or deleted.
Snapshot	What an ingredient looked like just before a purchase, saved with the purchase so it can be reversed.
Base unit	The unit an ingredient is stored in (for example kg).
Expected cash	What the drawer should hold, worked out from the records.
Rehearsal	A test run on the live database that always undoes itself.
Payout	Cash taken out of the open shift (a market run, gas, a supplier). It reduces expected cash.
Payout request	A cashier asking to take out cash above the limit. It counts only if the owner approves it.
Sale code	A random identifier the Till makes for each sale attempt, so the database can tell a repeat from a new sale.
Paper (late) entry	A sale written on paper during an outage, entered later and posted as a sale only when an owner approves it.
Dish price history	The list of every price a dish has had, with when each started.
Legacy (payment method)	The label on a supplier payment saved by the older screen, which could not say how it was paid.